WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI
POLITECHNIKI RZESZOWSKIEJ

Jan Drabek

Projekt aplikacji giełdy internetowej oraz aplikacji obciążającej

Praca dyplomowa magisterska

Opiekun pracy:
dr inż. Tomasz Rak

Rzeszów, 2024

1 Wstęp ... 5
1.1 Cel ... 5
1.2 Zakres ... 5
2 Technologie ... 7
2.1 Django ... 7
2.2 PostgreSQL ... 7
2.3 Celery ... 7
2.4 Locust ... 7
2.5 Docker ... 8
3 Implementacja[8] ... 11
3.1 Aplikacja giełdy ... 12
3.1.1 Diagram ERD ... 13
3.1.2 API ... 13
3.1.3 Operacje cykliczne ... 16
3.2 Aplikacja obciążeniowa ... 20
3.2.1 Diagram ERD ... 21
3.2.2 Klasy użytkowników ... 22
3.2.3 Pomiar wydajności w testach ... 27
4 Eksperymenty ... 32
4.1 Metodyka badań ... 32
4.1.1 Badania podstawowe ... 32
4.1.2 Badania z wykorzystaniem reguł asocjacyjnych [10] ... 33
4.1.3 Badania z wykorzystaniem klasyfikatorów ... 34
4.2 Testy ... 36
4.3 Analiza ... 39
4.4 Analiza z użyciem reguł asocjacyjnych ... 48
4.5 Analiza z użyciem klasyfikatorów ... 49
5 Podsumowanie ... 54
Literatura ... 57

1 Wstęp

Dynamiczny rozwój technologii informatycznych oraz cyfryzacja procesów gospodarczych znacząco wpływają na funkcjonowanie współczesnych rynków finansowych, stawiając przed nimi nowe wyzwania związane z automatyzacją i przetwarzaniem danych. Rosnące zapotrzebowanie na zautomatyzowane systemy transakcyjne oraz konieczność obsługi dużej liczby operacji w czasie rzeczywistym wymuszają poszukiwanie efektywnych i bezpiecznych rozwiązań technologicznych [1].

Niniejsza praca magisterska koncentruje się na weryfikacji wydajności systemu giełdowego, który umożliwia przetwarzanie danych w warunkach wysokiego obciążenia, z uwzględnieniem kluczowych wymagań współczesnych aplikacji finansowych, takich jak niezawodność i skalowalność. Istotnym elementem procesu projektowania aplikacji internetowych jest wybór odpowiednich technologii i narzędzi, które wspierają budowę rozwiązań zgodnych z wymaganiami nowoczesnych systemów transakcyjnych [2].

1.1 Cel

Celem niniejszej pracy jest zaprojektowanie i implementacja aplikacji giełdy internetowej, która umożliwi i efektywne przeprowadzanie transakcji handlowych w środowisku internetowym. Aplikacja ta jest wyposażona w funkcje umożliwiające zarządzanie kontami użytkowników, realizację zleceń kupna i sprzedaży oraz monitorowanie aktywności rynkowej w czasie rzeczywistym.

Równolegle, celem pracy jest również utworzenie aplikacji obciążającej, której zadania to symulacja ruchu użytkowników oraz dokonywanie intensywnych operacji na serwerach giełdy. Aplikacja ta posłużyła do testowania wydajności i skalowalności zaprojektowanego systemu, identyfikacji potencjalnych wąskich gardeł oraz oceny odporności systemu na różnego rodzaju obciążenia.

1.2 Zakres

Zakres pracy obejmuje następujące etapy:

1) Analiza wymagań: Identyfikacja kluczowych funkcjonalności aplikacji giełdy internetowej oraz aplikacji obciążającej na podstawie dostępnej literatury oraz analiz rynkowych.
2) Projekt systemu: Utworzenie szczegółowego projektu architektury systemu, obejmującego warstwę backendową, z uwzględnieniem zasad bezpieczeństwa i skalowalności.

3) Implementacja: Przeprowadzenie implementacji zaprojektowanych aplikacji przy użyciu technologii takich jak Django, Docker oraz PostgreSQL.
4) Testowanie: Przeprowadzenie wydajnościowych i obciążeniowych w celu weryfikacji poprawności działania systemu oraz jego zdolności do obsługi dużej liczby użytkowników i operacji. Następnie analiza wyników testów, do których wykorzystano, napisany w języku Python, framework Locust.

Praca zawiera pięć rozdziałów. W drugim rozdziale opisano użyte technologie. W trzecim przedstawiono szczegóły implementacji systemu. W czwartym rozdziale zaprezentowano sposób przeprowadzonych testów oraz analizę uzyskanych wyników. W ostatnim piątym rozdziale zawarte jest podsumowanie pracy.

2 Technologie

Do realizacji pracy użyto jednych z najczęściej używanych technologii do utworzenia aplikacji. Dzięki nim możliwa była implementacja aplikacji, dodanie obsługi asynchronicznych zadań, zaprojektowanie testów obciążających oraz konteneryzacja poszczególnych części całego sytemu.

2.1 Django

Django to framework aplikacji internetowych oparty na języku Python, który służył jako fundament projektu. Użyto go do tworzenia backendu aplikacji, zarządzania bazą danych, a także do definiowania modeli oraz logiki biznesowej. Django zapewnia szybki rozwój aplikacji, a także wbudowane narzędzia do obsługi autoryzacji, routingu, i integracji z bazami danych [3].

2.2 PostgreSQL

PostgreSQL to relacyjny system zarządzania bazami danych, który pełnił rolę głównego repozytorium danych dla aplikacji. Jego funkcje takie jak transakcje, indeksowanie, oraz możliwość pracy z dużymi ilościami danych sprawiły, że idealnie nadawał się do przechowywania i przetwarzania informacji związanych z giełdą oraz testami, zapewniając jednocześnie spójność i niezawodność [4].

2.3 Celery

Celery to narzędzie do zarządzania kolejkami zadań i ich asynchronicznej realizacji. W projekcie służył do obsługi operacji cyklicznych, takich jak przetwarzanie transakcji na giełdzie, aktualizacja statusu przeterminowanych ofert, obliczanie nowego kursu akcji oraz zmiana stanu finansów użytkowników. Umożliwił równoległe wykonywanie zadań oraz efektywne zarządzanie obciążeniem systemu. Do przechowywania danych o zadaniach w kolejce oraz zarządzania komunikacją między komponentami systemu wykorzystywany był Redis, pełniący rolę brokera wiadomości [5].

2.4 Locust

Locust to framework do testowania obciążeniowego, który został użyty do symulacji zachowania użytkowników aplikacji giełdowej. Pozwolił na zdefiniowanie różnorodnych scenariuszy testowych oraz pomiar wydajności API pod kątem czasu odpowiedzi i liczby

przetworzonych zapytań. Dzięki niemu możliwe było zbadanie wydajności systemu pod różnym obciążeniem [6].

2.5 Docker

Docker to platforma do konteneryzacji, która umożliwiła łatwą izolację poszczególnych komponentów systemu. Dzięki Dockerowi aplikacja mogła być uruchamiana w zdefiniowanym środowisku, niezależnie od platformy. Konteneryzacja ułatwiła zarządzanie i wdrażanie różnych usług w sposób skalowalny i elastyczny [7].

Rys. 2.1 przedstawia plik Dockerfile, który jest używany do zbudowania obrazu aplikacji giełdy. W pierwszej linii pliku zdefiniowano bazowy obraz systemu, który wykorzystuje Python w wersji 3.10-slim. Jest to lekka wersja Pythona, co zmniejsza rozmiar finalnego obrazu. Następnie za pomocą instrukcji WORKDIR ustalono katalog roboczy w kontenerze jako /app, do którego w kolejnych krokach kopiowane są wymagane do zainstalowania biblioteki oraz pliki aplikacji. Komenda RUN uruchamia instalację zależności z pliku requirements.txt. Ostatecznie komenda CMD definiuje polecenie, które uruchamia serwer aplikacji za pomocą Gunicorn, wskazując na plik wsgi, gdzie znajduje się punkt wejścia do aplikacji Django. Podobne pliki dockerfile zostały utworzone dla aplikacji obciążeniowej oraz dla kontenera zbierającego dane o zużyciu procesora i pamięci RAM.

```
stockProject > Dockerfile
1  FROM python:3.10-slim
2  WORKDIR /app
3  COPY requirements.txt /app/
4  RUN pip install --no-cache-dir -r requirements.txt
5  COPY . /app/
6  CMD ["gunicorn", "--bind", "0.0.0.0:8080", "stockProject.wsgi:application"]
```

Rys. 2.1 Dockerfile aplikacji giełdy

Rys. 2.2 przedstawia fragment pliku docker-compose.yml, który jest odpowiedzialny za definiowanie usług aplikacji giełdy oraz jej zależności w formie kontenerów. W sekcji services zdefiniowano serwis o nazwie web, który buduje aplikację na podstawie pliku Dockerfile. W docker-compose zostały zadeklarowane następujące elementy:

1) Ścieżka do budowy obrazu.
2) Komenda zawierająca wszystkie operacje konieczne do wykonania przed pierwszym uruchomieniem aplikacji giełdy. Są to: utworzenie migracji, wykonanie ich, dodanie plików statycznych oraz uruchomienie serwera.
3) Katalog roboczy aplikacji.
4) Port.

5) Zależności.
6) Zmienne środowiskowe.
7) Nazwa sieci.

```yaml
stockProject > docker-compose.yml
services:
  web:
    build: .
    command:
      sh -c
      "python manage.py makemigrations &&
      python manage.py migrate &&
      python manage.py migrate --database=test &&
      python manage.py collectstatic --noinput &&
      gunicorn stockProject.wsgi:application --bind 0.0.0.0:8080"
    volumes:
      - .:/app
    ports:
      - "8080:8080"
    depends_on:
      - db
      - db_test
    environment:
      ENV_ID: "web"
      TRANSACTION_TIME: "15.0"
    networks:
      - gielda_network
```

Rys. 2.2 Fragment pliku docker-compose.yml

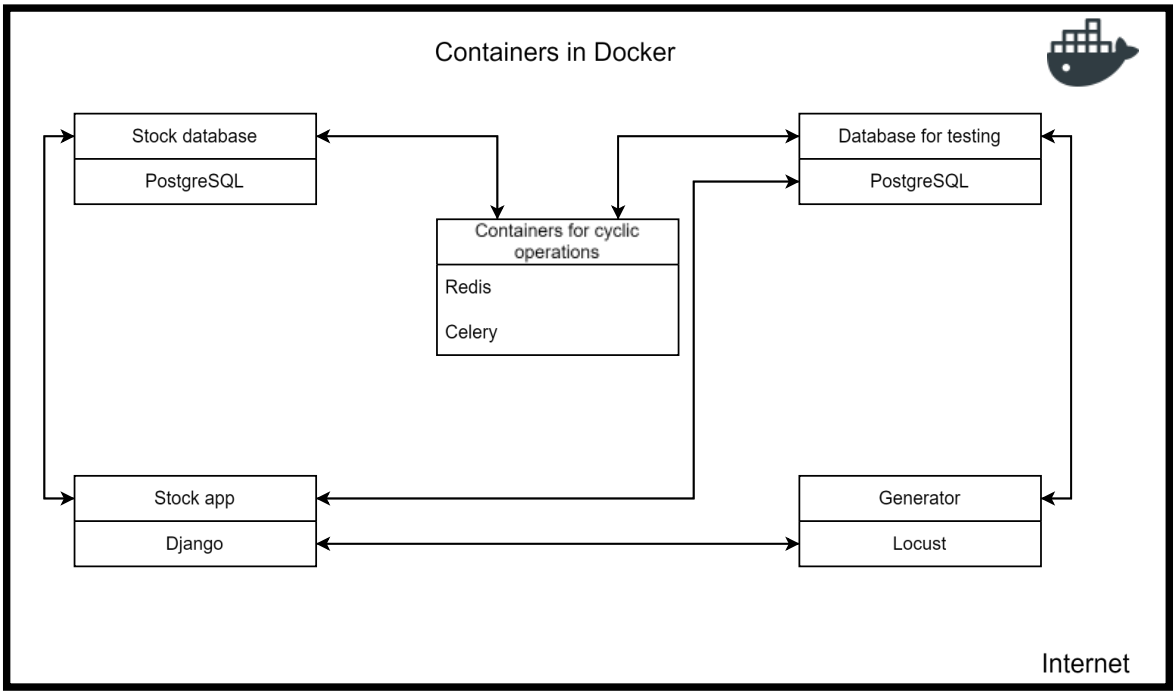
W pliku znalazły się również serwisy:

1) Generatora ruchu (locust).
2) Bazy danych giełdy (db). Działa na porcie 6543:5432.
3) Bazy danych do testów (db_test). Działa na porcie 6544:5432.
4) Redisa (redis). Działa na porcie 6379:6379.
5) Celery beat do harmonogramowania zadań (celery_beat).
6) Celery worker do aktualizacji akcji firm (celery_worker_stock_rates).
7) Celery worker do grupowania transakcji (celery_worker_schedule_transactions).
8) Celery worker do realizacji transakcji (celery_worker_execute_transactions). Powstaje wskazana liczba replik, zależna od specyfikacji testu.
9) Celery worker do zmieniania statusu przeterminowanych ofert (celery_worker_expire_offers).
10) Celery worker aktualizujący salda użytkowników (celery_worker_balance_updates).
11) Kontenera do monitorowania zużycia procesora i pamięci RAM (monitor).

Aplikacja giełdy jest zależna od serwisów baz danych. Serwis monitorujący zużycie procesora jest zależny od bazy danych do testów. Generator ruchu do działania serwisu giełdy oraz baz danych. Wszystkie kontenery wykonujące zadania Celery są zależne od serwisu Redisa, bazy danych giełdy oraz bazy danych testu. Do wykonywania zadań w Celery konieczny jest broker. W tym celu używany jest Redis. Wymienione w tym akapicie serwisy zapisują i odczytują dane z baz danych, dlatego są od nich zależne.

3 Implementacja[8]

Pierwszym krokiem w pracy nad systemem było zaprojektowanie modelu bazy danych. Miało to na celu uniknięcie sytuacji, w której konieczne byłoby dodawanie nowych tabel i relacji, co powoduje zmiany w kodzie. Nie udało się całkowicie uniknąć tego problemu. W trakcie implementacji nie przewidziano, że równoczesna zmiana pieniędzy posiadanych przez użytkownika, przez kilka różnych zapytań do API powoduje, że wartość w tym polu nie jest prawidłowa. Aby to rozwiązać dodano tabelę BalanceUpdate, która zbiera wszystkie zmiany w bilansie dla poszczególnych użytkowników. Kolejnym etapem było zakodowanie backendu aplikacji giełdy. Głównym jej zadaniem była obsługa dodawania ofert kupna i sprzedaży oraz przechowywanie informacji o firmach i użytkownikach. Następnie zabrano się za dodanie operacji, które są wykonywane cyklicznie. Po zakończeniu pracy nad giełdą, przystąpiono do pracy nad generatorem ruchu, którego zadaniem było symulowanie zachowania użytkowników oraz obciążanie aplikacji giełdy. Do realizacji tego zadania użyto frameworka Locust. Zastosowano inną strategię i najpierw napisano kod obciążnika, a dopiero potem dodano do niego bazę danych, której zadaniem było zbieranie danych z testów. Na Rys. 3.1 został przedstawiony schemat zaprojektowanego systemu giełdy.



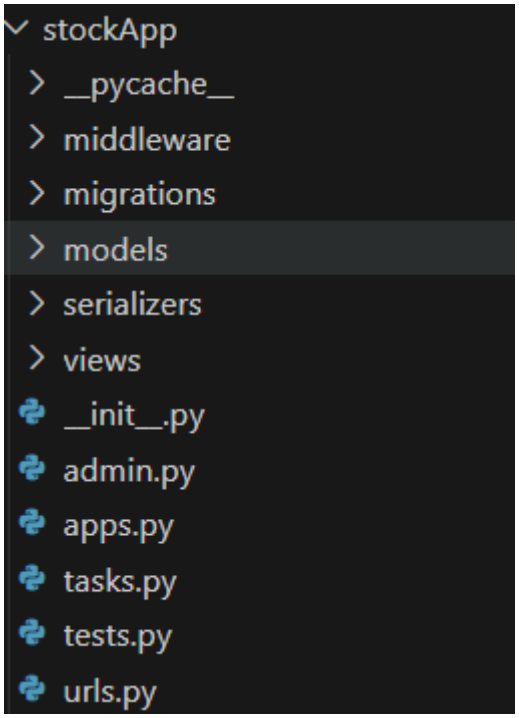
Rys. 3.1 Schemat systemu

3.1 Aplikacja giełdy

Funkcjonalności giełdy były wzorowane na innych rozwiązaniach [9,10]. Użytkownik przed rozpoczęciem udziału w giełdzie musi się zarejestrować i zalogować. Następnie otrzymuje dostęp do dodawania firmy, dodawania i usuwania ofert kupna oraz sprzedaży, przeglądania swoich ofert, swoich udziałów w firmach, informacji o swoim koncie, listy dostępnych firm oraz ich aktualnych akcji.

W aplikacji przyjęto podział na foldery przedstawiony na Rys. 3.2. Pliki .py oprócz task.py, który implementuje zadania realizowane cyklicznie, to pliki automatycznie dodane przez Django. W przedstawionej strukturze folderów zawarto:

1) Middleware - warstwa oprogramowania pośredniczącego pomiędzy żądaniem od użytkownika a widokiem aplikacji. Działa jako filtr, który przetwarza żądania i odpowiedzi HTTP. Umożliwiło dodanie danych w trakcie testów, bez konieczności modyfikacji logiki widoków.
2) Migrations – migracje przeprowadzone przez Django.
3) Models – modele obiektów bazy danych.
4) Serializers – odpowiada za serializację i deserializację danych z API, co ułatwia pracę z bazą danych.
5) Views – obsługa żądań API.

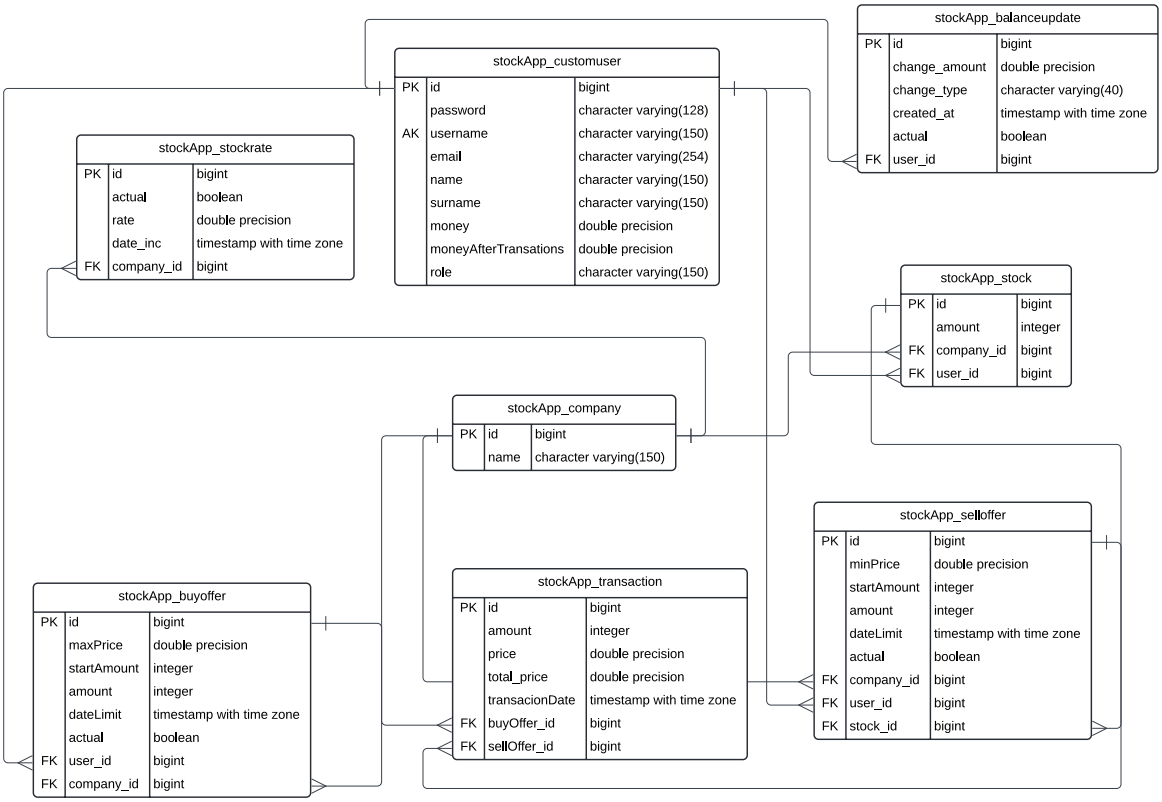


Rys. 3.2 Struktura plików aplikacji giełdy

3.1.1 Diagram ERD

Diagram na Rys. 3.3 przedstawia modele bazy danych giełdy. Kod opisujący modele znajduje się w folderze models przedstawionym na Rys. 3.2. Schemat zawiera tabele:

1) Customuser – informacje o użytkowniku.
2) Balanceupdate – zmiana stanu finansów użytkownika.
3) Stockrate – informacja o cenie akcji danej firmy.
4) Stock – zawiera dane o akcjach posiadanych przez użytkowników.
5) Company – tabela dotycząca firm.
6) Buyoffer – przechowuje informacje o ofertach kupna.
7) Selloffer – tabela z ofertami sprzedaży.
8) Transaction – reprezentuje dane dotyczące transakcji.

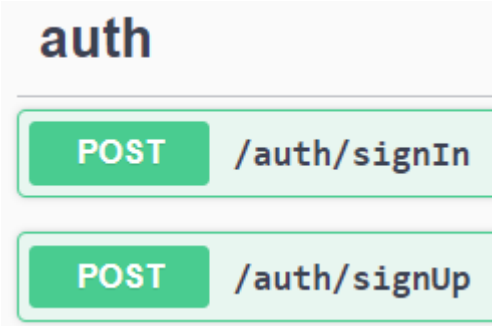


Rys. 3.3 Diagram ERD aplikacji giełdowej

3.1.2 API

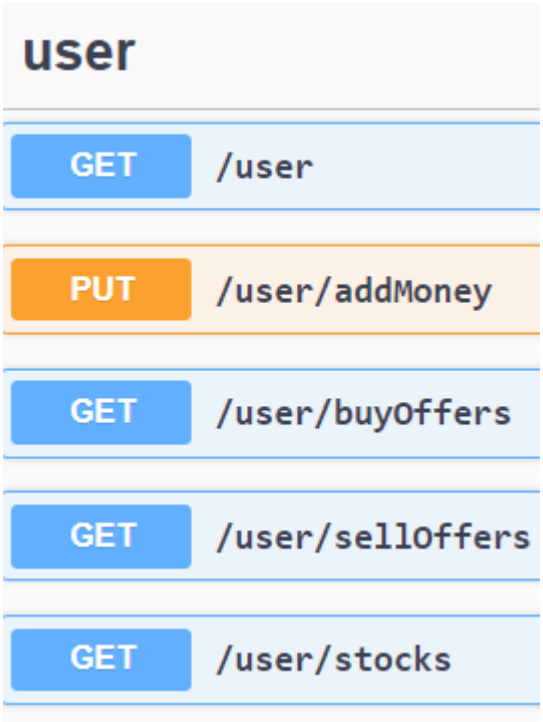
Głównym składnikiem aplikacji są endpoint'y. Do opisu wykorzystano Swagger. W strukturze projektu znajdują się w folderze views zaprezentowanym na Rys. 3.2. Na Rys. 3.4 przedstawiono punkty końcowe służące do uwierzytelniania użytkownika. Przy rejestracji nowego użytkownika zwracany jest komunikat o sukcesie lub niepowodzeniu

tego zdarzenia. Po logowaniu użytkownik dostaje Django rest framework token, który jest niezbędny przy wszystkich innych żądaniach.



Rys. 3.4 Endpointy uwierzytelniania

Następnym krokiem było utworzenie funkcjonalności dotyczących użytkownika oraz firm. Dla firm uwzględniono wyświetlenie listy wszystkich firm, dodanie nowej firmy. Natomiast użytkownik może odczytywać i zmieniać wszelkie swoje dane oraz zarządzać swoimi ofertami. Na Rys. 3.5 oraz Rys. 3.6 zaprezentowano punkty końcowe realizujące powyższe założenia.



Rys. 3.5 Endpointy użytkownika

company

POST /company/addCompany

GET /company/companies

GET /company/getCompaniesStockRates

Rys. 3.6 Endpointy firmy

Esencją całego sytemu jest składanie ofert kupna i sprzedaży, następnie na ich podstawie przeprowadzenie transakcji. Na Rys. 3.7 pokazano kontrolery do tworzenia oraz usuwania ofert.

offer

POST /offer/addBuyOffer

POST /offer/addSellOffer

DELETE /offer/deleteBuyOffer/{id}

DELETE /offer/deleteSellOffer/{id}

Rys. 3.7 Endpointy ofert

Implementacja punktu końcowego /offer/addBuyOffer została przedstawiona na Rys. 3.8. W linii 13 następuje przekazania danych do serializera. Każdy endpoint ma swój dedykowany serializer znajdujący się w folderze Jeżeli dane są poprawne to zapisujemy je w bazie danych i zwracamy informacje o sukcesie. W innym wypadku przekazujemy napotkane błędy. Implementacja pozostałych punktów końcowych jest bardzo podobna.

```
10  @api_view(['POST'])
11  @permission_classes([IsAuthenticated])
12  def addBuyOffer(request):
13      serializer = BuyOfferSerializer(data=request.data, context={'request': request})
14      if serializer.is_valid():
15          serializer.save()
16          return Response(serializer.data, status=status.HTTP_201_CREATED)
17      return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Rys. 3.8 Implementacja endpointu /offer/addBuyOffer

3.1.3 Operacje cykliczne

Do implementacji transakcji, aktualizacji akcji firm, zmieniania statusu przeterminowanych ofert oraz zmiany stanu konta użytkowników wykorzystano narzędzie Celery z brokerem wiadomości Redis. Implementacja wyżej wymienionych zadań znajduje się w pliku tasks.py. W pliku settings.py, znajdującym się w głównym folderze projektu, zawierającym wszystkie ustawienia Django konieczne było dodanie ustawień Celery.

Pierwotnie testowano działanie systemu bez podziału na widoczne na Rys. 3.9 dedykowane kolejki, co jednak skutkowało znacznym spadkiem wydajności aplikacji. Rozdzielenie zadań na poszczególne kolejki pozwoliło na bardziej efektywne przetwarzanie zadań oraz uniknięcie przeciążeń, które miały miejsce, gdy wszystkie zadania były bez większej kontroli przydzielane przez Celery beat do workerów.

Konfiguracja w pliku settings.py definiuje pięć kolejek: default, transactions, balance_updates, stock_rates oraz expire_offers. W ten sposób możliwe jest przypisanie każdemu workerowi Celery specyficznego zadania, co pozwala na lepsze rozłożenie obciążenia oraz ograniczenie przeciążeń na jednym procesie. Każda z wymienionych kolejek odpowiada za konkretne zadania:

1) default – kolejka domyślna. Trafiają tu zadania nieprzypisane do innych kolejek.
2) transactions – obsługuje zadania związane z realizacją transakcji na giełdzie.
3) balance_updates – odpowiedzialna za aktualizacje stanów kont użytkowników.
4) stock_rates – zajmuje się aktualizacją cen akcji.
5) expire_offers – obsługuje proces zmiany statusu przeterminowanych ofert.

```
171  from kombu import Exchange, Queue
172
173  CELERY_QUEUES = (
174      Queue('default', Exchange('default'), routing_key='default'),
175      Queue('transactions', Exchange('transactions'), routing_key='transactions'),
176      Queue('balance_updates', Exchange('balance_updates'), routing_key='balance_updates'),
177      Queue('stock_rates', Exchange('stock_rates'), routing_key='stock_rates'),
178      Queue('expire_offers', Exchange('expire_offers'), routing_key='expire_offers'),
179  )
```

Rys. 3.9 Implementacja kolejek Celery w pliku settings.py

Na Rys. 3.10 zaprezentowano fragment pliku celery.py, w którym znajduje się konfiguracja harmonogramu wykonywanych operacji. W widocznym fragmencie kodu pokazano proces tworzenia zadania cyklicznego, które będzie zarządzane przez Celery Beat. To narzędzie działa jako menedżer harmonogramu i jest odpowiedzialne za planowanie oraz przekazywanie zadań do workerów, co zapewnia regularne wykonywanie operacji w systemie.

Konfiguracja rozpoczyna się od podania zadania oraz ścieżki do funkcji, która ma zostać wykonana – w tym przypadku jest to stockApp.tasks.schedule_transactions. Funkcja ta jest odpowiedzialna za inicjację transakcji. Kolejnym istotnym elementem jest określenie częstotliwości wykonywania operacji. W linii 23 wprowadzono wartość, która odpowiada za interwał czasowy pomiędzy kolejnymi wykonaniami zadania. Jest ona pobierana dynamicznie ze zmiennej środowiskowej TRANSACTION_TIME za pomocą funkcji os.getenv. Na koniec w linii 24 wskazujemy kolejkę, do której przypisane jest zadanie. Schemat konfiguracji dla wszystkich zdarzeń jest taki sam.

```
20  app.conf.beat_schedule = {
21      'schedule_transactions': {
22          'task': 'stockApp.tasks.schedule_transactions',
23          'schedule': float(os.getenv("TRANSACTION_TIME")),
24          'options': {'queue': 'transactions'},
25      },
```

Rys. 3.10 Fragment harmonogramu implementujący zadanie schedule_transactions z pliku celery.py

Na podstawie analizy diagramu transakcji widocznego na Rys. 3.11, proces realizacji transakcji można szczegółowo opisać w kilku kluczowych etapach.

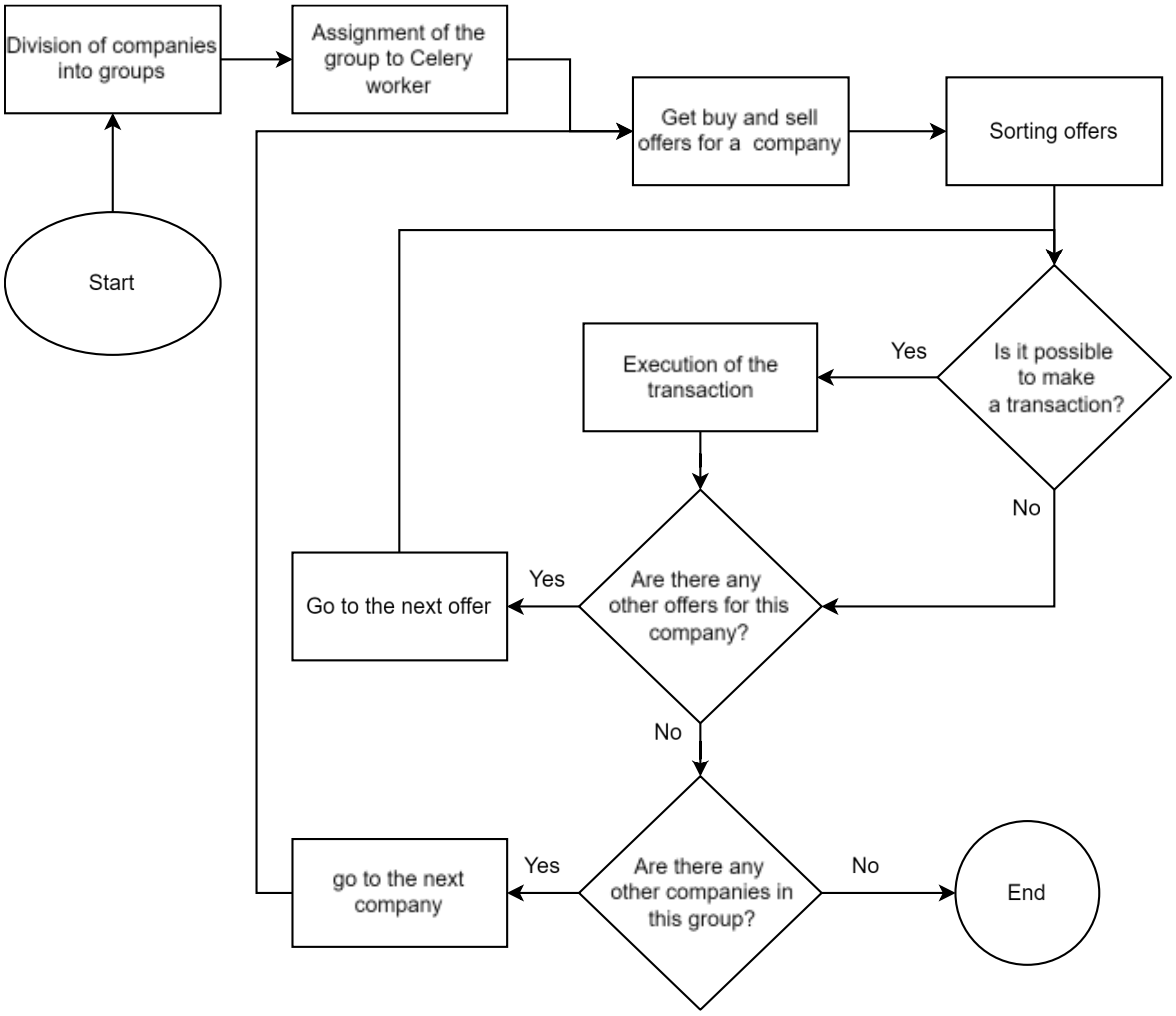
Pierwszym krokiem w przedstawionym schemacie jest inicjacja procesu, która następuje po otrzymaniu sygnału z narzędzia Celery beat. Sygnał ten aktywuje workera, którego zadaniem jest wywołanie funkcji schedule_transactions. W momencie jej uruchomienia, następuje podział firm na grupy, co pozwala na optymalizację procesów

transakcyjnych i lepsze zarządzanie czasem obliczeń. Liczba grup zależy od ilości firm i jest równa największej liczbie naturalnej mniejszej lub równej pierwiastkowi z liczby firm. Każda grupa firm jest przetwarzana osobno, a operacje transakcyjne mogą być równolegle obsługiwane przez różnych workerów.

W drugim etapie jeden z workerów odpowiedzialnych za realizację transakcji otrzymuje listę identyfikatorów firm, które zostały mu przydzielone do przetworzenia przez Celery beat. Na tym etapie worker ten rozpoczyna iterację po firmach oraz ich ofertach. Sortowanie odbywa się w taki sposób, że oferty sprzedaży są sortowane od najwyższej ceny, a oferty kupna od jak najniższej. Ta strategia próbuje zadowolić zarówno kupujących jak i sprzedających.

Następnie ten sam worker przechodzi do procesu realizacji transakcji. W przypadku gdy, wszystkie warunki do przeprowadzenia transakcji pomiędzy dwoma ofertami są spełnione, to transakcja zostaje miedzy nimi wykonywana. Mechanizm ten umożliwia wymianę aktywów lub usług zgodnie z parametrami określonymi w ofertach. Ważnym elementem tego procesu jest uwzględnienie, czy kupujący posiada środki na realizację transakcji, a sprzedający nadal posiada akcje danej firmy.

Po zakończeniu przetwarzania wszystkich ofert jednej firmy, worker przechodzi do kolejnej firmy z listy, kontynuując ten proces, aż wszystkie firmy w danej grupie zostaną obsłużone. Dzięki zastosowaniu iteracyjnego podejścia oraz przemyślanemu podziałowi firm, cały proces realizacji transakcji jest zoptymalizowany pod względem wydajności i efektywności. Zakończenie przetwarzania transakcji oznacza przejście workera do oczekiwania na kolejne zadanie, które może być uruchomione ponownie po otrzymaniu kolejnego sygnału z Celery beat.



Rys. 3.11 Schemat transakcji

Na schemacie nie sposób było przedstawić wszystkie warunki, które muszą zostać spełnione, aby mogła nastąpić transakcja. Natomiast widać je w kodzie zaprezentowanym na Rys. 3.12. W zaprezentowanym fragmencie kodu widoczna jest pętla, która iteruje po dwóch rodzajach ofert: ofertach kupna oraz ofertach sprzedaży. Celem tej pętli jest znalezienie pary ofert, które spełniają odpowiednie warunki, umożliwiające dokonanie transakcji. Warunki te są szczegółowo określone w kodzie, a każde z nich ma określoną rolę w procesie dopasowania ofert i zawierania transakcji.

Pierwszym istotnym elementem weryfikacji jest sprawdzenie, czy oferta kupna i oferta sprzedaży dotyczą tej samej firmy: buy_offer.company == sell_offer.company. Ten warunek wydaje się na pierwszy rzut oka redundantny, ponieważ wcześniej w procesie selekcji ofert przyjęto, że oferty dotyczą tej samej firmy. Jednak wprowadzenie takiej weryfikacji może być formą zabezpieczenia przed potencjalnymi błędami wynikającymi z nieprzewidzianych sytuacji w trakcie iteracji po firmach. Wprowadzenie tego warunku może poprawić stabilność i odporność systemu na anomalie.

Kolejne dwa warunki dotyczą ilości, jaką obejmuje oferta: buy_offer.amount > 0 and sell_offer.amount > 0. Tu podobnie jak przy warunku dotyczącym sprawdzenia firm, jest to forma zabezpieczenia przed niechcianym zachowaniem. Przy prawidłowym działaniu takie oferty mają ustawione pole actual na false i powinny być odrzucone na wcześniejszym etapie.

Kolejny warunek: buy_offer.maxPrice >= sell_offer.minPrice, jest absolutnie kluczowy dla zawarcia transakcji. Weryfikuje on, czy cena maksymalna, jaką jest gotów zapłacić kupujący, jest większa lub równa minimalnej cenie, którą akceptuje sprzedający. To właśnie ten warunek stanowi ekonomiczne jądro mechanizmu dopasowywania ofert.

Ostatni warunek: seler != buyer, zapewnia, że nie zostanie zawarta transakcja pomiędzy jedną osobą. Choć niektóre z tych warunków mogłyby wydawać się nadmiarowe, to ich obecność podkreśla dbałość o szczegóły i stabilność systemu, co jest szczególnie ważne w kontekście złożonego mechanizmu rynkowego.

Jak widać w linii 38 do przeprowadzenia transakcji uwzględniana jest mniejsza liczba akcji z dostępnych w ofercie kupna i sprzedaży. Jeżeli obie oferty dotyczą tej samej ilości akcji to każda z nich zostanie w pełni zrealizowana i ich flaga actual zmienia wartość na false. Gdy wartości dostępnych akcji w ofertach są różne to ta z mniejszą ilością zostaje całkowicie zrealizowana.

```
29                  for buyOffer in buyOffers:
30                      for sellOffer in sellOffers:
31                          buyer = buyOffer.user
32                          seller = sellOffer.user
33                          if (buyOffer.company == sellOffer.company and
34                              buyOffer.amount > 0 and
35                              sellOffer.amount > 0 and
36                              buyOffer.maxPrice >= sellOffer.minPrice and
37                              seller != buyer):
38                              amountToTrade = min(buyOffer.amount, sellOffer.amount)
```

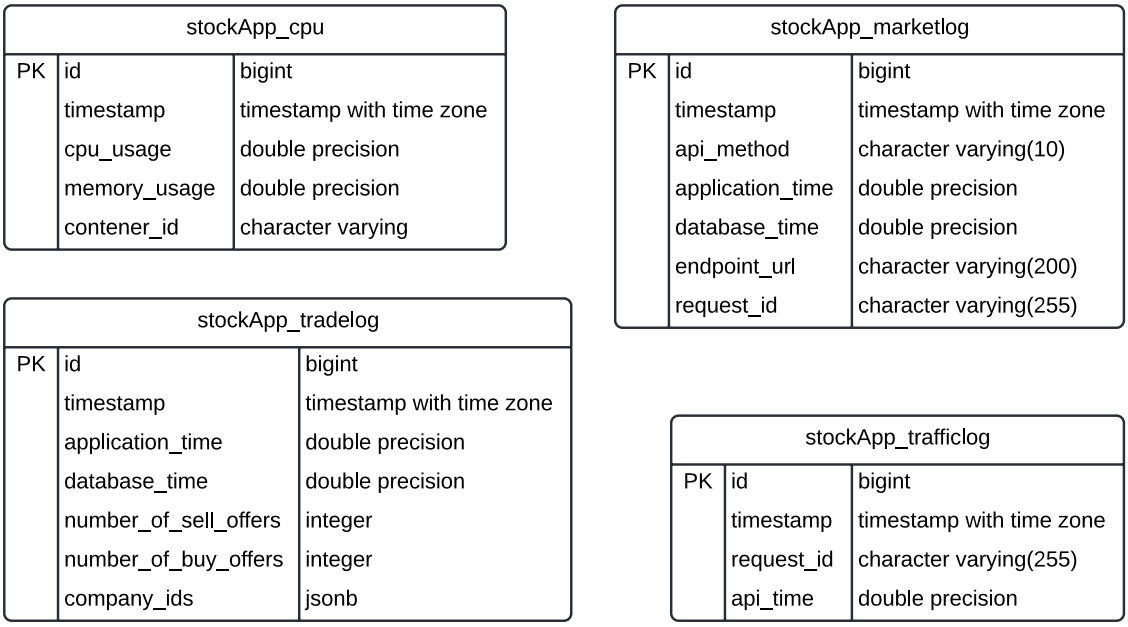
Rys. 3.12 Warunki konieczne do realizacji transakcji. Fragment kodu z pliku tasks.py

3.2 Aplikacja obciążeniowa

Aby przetestować wydajność giełdy został zaimplementowany generator zapytań. Jego głównym celem było symulowanie zachowania użytkowników oraz zebranie czasów zapytań do API w tabeli. Do jego budowy została użyta biblioteka Locust, które daje szeroki wybór parametrów do zastosowania w testach.

3.2.1 Diagram ERD

Diagram ERD przedstawiony na Rys. 3.13 obrazuje strukturę bazy danych, która była kluczowym elementem systemu zbierania danych z testów obciążeniowych. Składa się z czterech tabel: cpu, tradelog, marketlog oraz trafficlog. Każda z tych tabel pełni określoną funkcję w kontekście monitorowania i rejestrowania zachowań systemu podczas testów.



Rys. 3.13 Diagram ERD bazy danych do testów

Tabela cpu przechowuje dane dotyczące zużycia zasobów systemowych takich jak procesor oraz pamięć RAM. Kluczowym elementem tej tabeli jest kolumna contener_id, która identyfikuje kontener, z którego pobierane są dane. Pomiary są wykonywane cyklicznie co dwie sekundy przez dedykowany do tego kontener, co pozwala na dokładne śledzenie zmian w wydajności systemu.

Tabela tradelog jest centralnym elementem monitorowania wydajności systemu transakcyjnego. Zawiera informacje o czasie potrzebnym na przetwarzanie transakcji, rozdzielonym na dwa komponenty: czas aplikacji oraz czas bazy danych. Dodatkowo zapisuje liczbę ofert kupna i sprzedaży oraz listę obsłużonych firm, co umożliwia analizę obciążenia systemu na poziomie operacyjnym. Operacja zapisu danych wykonywana jest przez Celery worker odpowiedzialny za przetwarzanie transakcji.

Tabela marketlog przechowuje informacje o zapytaniach API związanych z aplikacją giełdową. Pola api_method i endpoint_url pozwalają na dokładną analizę, jakie żądania API były przetwarzane oraz w jakim czasie. Tak jak w przypadku tradelog mierzony jest czas aplikacji oraz bazy danych. Pomiar danych jest wykonywany w aplikacji giałdy. Zawartość

tabeli market jest ściśle powiązana z tabelą trafficlog, w której rejestrowane są czasy zapytań generowanych przez obciążnik.

Trafficlog stanowi integralny element systemu testów obciążeniowych. Dane do tej tabeli są dodawane przez generator zapytań. Rejestrowany jest jedynie czas odpowiedzi API na poziomie sieci, wraz z unikalnym identyfikatorem zapytania, który umożliwia późniejsze połączenie go z odpowiednim wpisem w tabeli marketlog. Możliwe było zapisanie danych z marketlog oraz trafficlog w jednej tabeli. Natomiast wiązałoby się to z koniecznością przesyłania dodatkowych danych przez API. W zamian za to byłby tylko jeden zapis do bazy danych. Do ustalenia, które podejście daje większą wydajność konieczne byłyby dodatkowe testy.

3.2.2 Klasy użytkowników

Podczas przeprowadzania testów wydajności aplikacji giełdowej kluczowym aspektem było utworzenie odpowiednich profili użytkowników symulujących różnorodne zachowania graczy na rynku. Takie podejście miało na celu jak najwierniejsze odwzorowanie aktywności rzeczywistych użytkowników platformy, co jest istotne zarówno z perspektywy testowania obciążenia, jak i optymalizacji działania całego systemu.

W ramach testów utworzono trzy klasy użytkowników: aktywnego gracza, przeglądającego oraz gracza aktywnego z analizą rynku. Symulowanie zachowań graczy pozwalało na monitorowanie wydajności systemu w różnych warunkach oraz testowanie scenariuszy, które mogą wystąpić w rzeczywistych sytuacjach rynkowych.

Na Rys. 3.14 przedstawiono schemat zachowania aktywnego gracza w trakcie testów obciążeniowych. Proces rozpoczyna się od zarejestrowania użytkownika, logowania do systemu oraz utworzenia jednej firmy, w której gracz na początku ma wszystkie udziały. Działania te stanowią wstępny krok, umożliwiający dalszą interakcję z platformą.

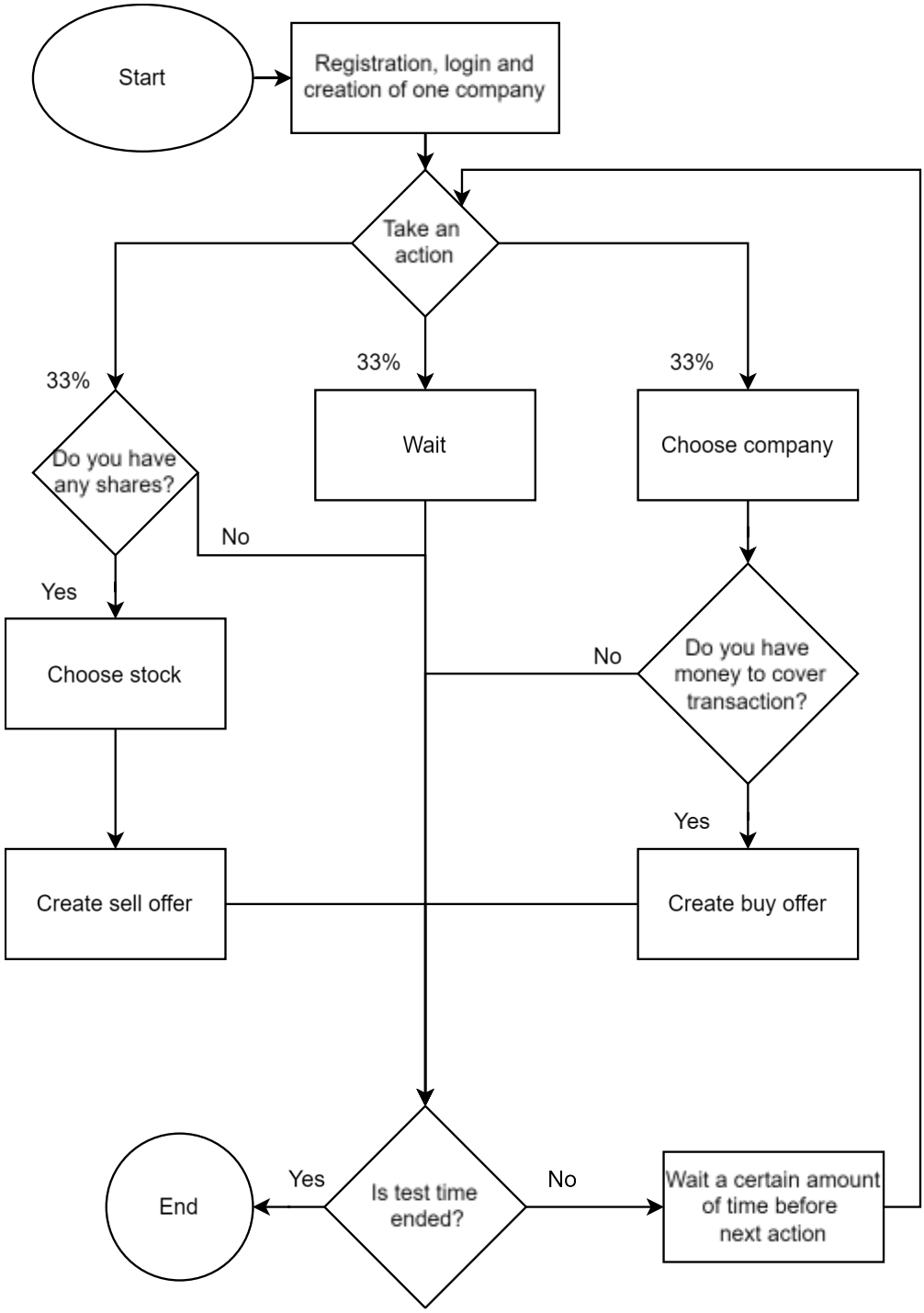
Następnie użytkownik musi podjąć decyzję o wykonaniu konkretnej akcji. W schemacie mamy trzy możliwości działania, każda z równym prawdopodobieństwem. Pierwsza opcja to oczekiwanie, co odzwierciedla sytuację, w której użytkownik nie podejmuje żadnych działań przez pewien czas, symulując mniej aktywne podejście do rynku.

Drugą możliwością jest złożenie oferty sprzedaży. Aby to zrobić, użytkownik musi posiadać akcje jakiejś spółki. Jeśli warunek ten jest spełniony, użytkownik losowo wybiera akcje jednej z firm, w której ma udziały i tworzy ofertę sprzedaży.

Trzecia opcja polega na złożeniu oferty kupna. Tutaj kluczowym warunkiem jest posiadanie wystarczających środków finansowych na pokrycie transakcji. Jeśli użytkownik

ma odpowiednią ilość środków zostaje wybrana firma. Następnie tworzona jest odpowiednia oferta kupna.

Po wykonaniu dowolnej akcji, test sprawdza, czy czas testu dobiegł końca. Jeśli nie, użytkownik czeka określony czas przed podjęciem kolejnej akcji. Taki mechanizm pozwala na symulowanie bardziej realistycznych zachowań rynkowych, gdzie użytkownicy nie dokonują transakcji bez przerwy, lecz wykonują je w określonych interwałach czasowych.



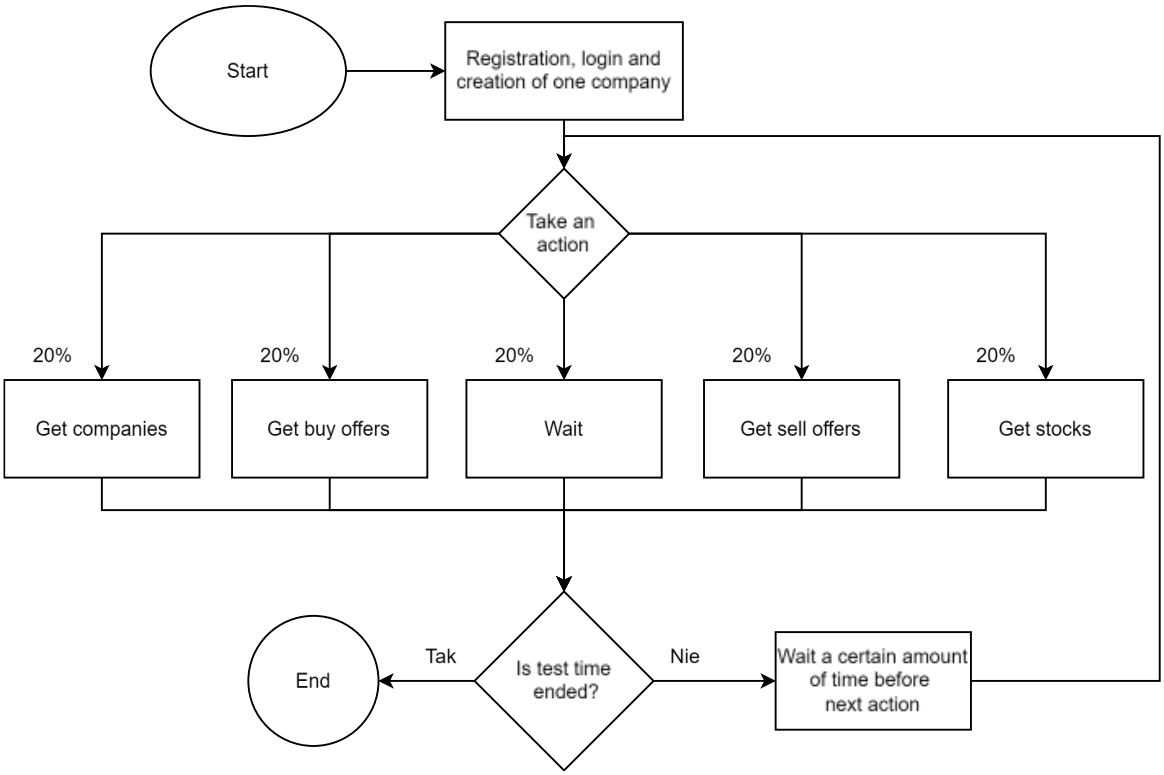
Rys. 3.14 Schemat zachowania użytkownika aktywnego

Drugą klasą użytkownika jest gracz przeglądający. W odróżnieniu od gracza aktywnego, użytkownik przeglądający nie podejmuje transakcji, lecz przegląda dostępne dane. Na Rys. 3.18 przedstawiono schemat zachowania gracza przeglądającego. Proces rozpoczyna się tak samo jak u gracza aktywnego od rejestracji.

Po rejestracji użytkownik musi podjąć decyzję o wykonaniu konkretnej akcji. W tym przypadku możliwe jest wykonanie jednej z pięciu czynności, przy czym każda z nich ma równe prawdopodobieństwo:

1) Pobranie listy firm.
2) Pobranie ofert kupna.
3) Oczekiwanie.
4) Pobranie ofert sprzedaży.
5) Pobranie akcji.

Po wykonaniu jednej z wyżej wymienionych akcji, podobnie jak w przypadku gracza aktywnego, system sprawdza, czy czas przeznaczony na test dobiegł końca. Jeśli tak, test zostaje zakończony, jeśli nie, użytkownik oczekuje określony czas, po czym podejmuje kolejną akcję.



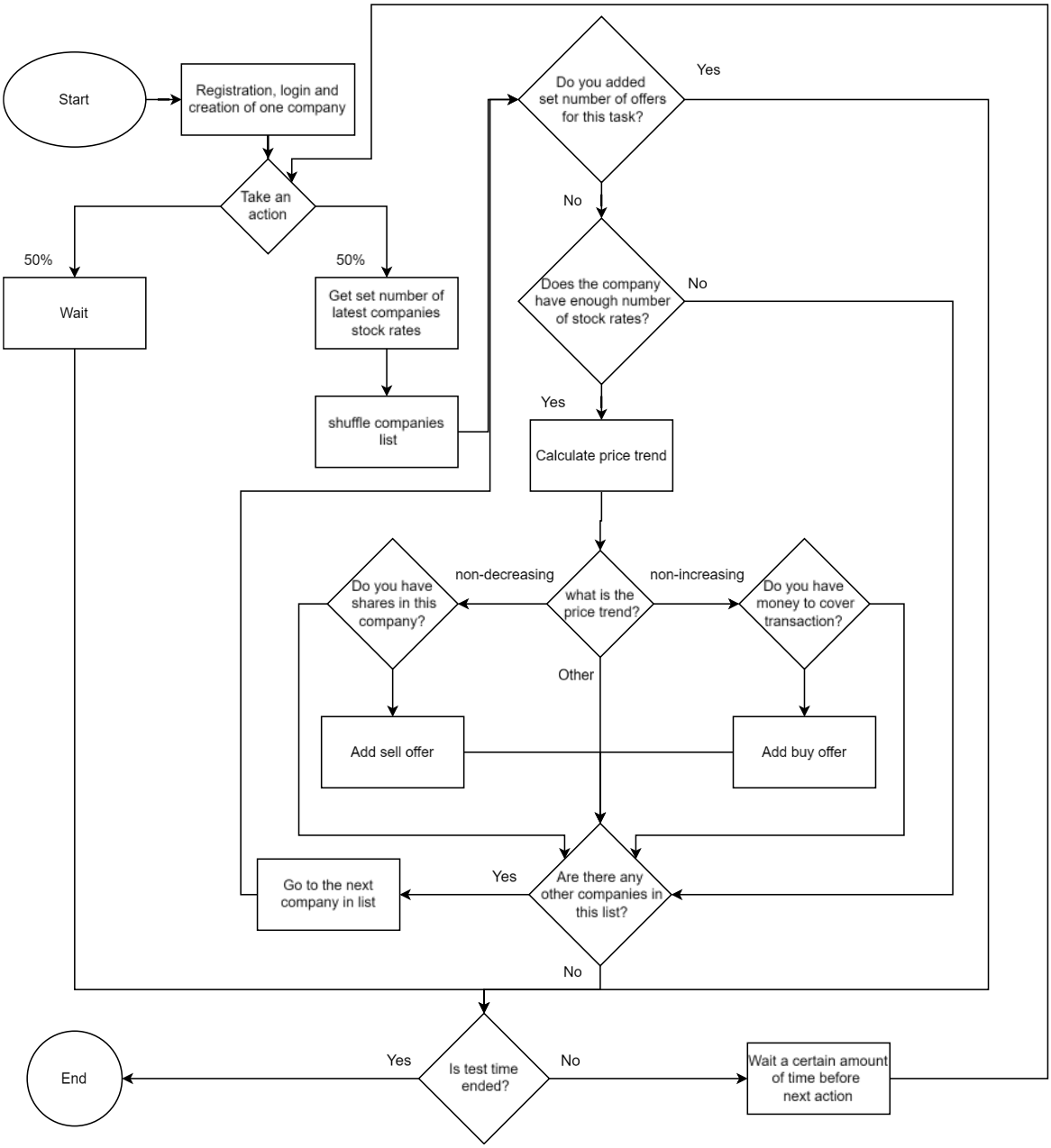
Rys. 3.15 Schemat zachowania użytkownika przeglądającego

Ostatnią zaimplementowaną klasą jest użytkownik aktywny z analizą rynku. Jego schemat przedstawiono na Rys. 3.16. Gracz ten, w odróżnieniu od innych typów użytkowników, nie podejmuje decyzji losowo, lecz wykorzystuje notowania firm do ustalenia aktualnie panujących trendów cenowych. Na ich podstawie podejmuje decyzje dotyczące składanych ofert. Ten sposób działania skutkuje większą szansą na zysk od klasy aktywnego gracza.

Gracz z analizą rynku działa w dwóch głównych fazach: fazie analizy oraz fazie działania. W pierwszej kolejności użytkownik sprawdza, czy firma, której akcje chce analizować, posiada wystarczającą liczbę historycznych notowań, co jest konieczne do przeprowadzenia analizy trendu. Jeżeli notowań jest zbyt mało, firma jest pomijana w danej sesji analizy. Notowania te powstają jedynie wtedy, gdy na rynku są oferty, co oznacza, że aktywność innych użytkowników, takich jak gracze losowo składający oferty kupna i sprzedaży, jest konieczna. W innym wypadku nie będzie nowych notowań i gracz z analizą rynku nie wystawi żadnej oferty.

W fazie działania, na podstawie zebranych danych, użytkownik podejmuje decyzję o zakupie bądź sprzedaży akcji. Jeśli trend cen dla danej firmy wykazuje ciąg niemalejący, użytkownik postrzega to jako sygnał do sprzedaży akcji. Kluczowym warunkiem jest jednak posiadanie odpowiedniego pakietu akcji przez użytkownika, co ogranicza liczbę możliwych ofert sprzedaży. Natomiast, jeśli trend cen wskazuje na ciąg nierosnący, gracz podejmuje decyzję o zakupie akcji, licząc na przyszłe zyski w wyniku wzrostu cen.

Ważnym elementem symulacji jest także limit zapytań, które gracz może wykonać w jednym cyklu. Bez tego ograniczenia, liczba ofert kupna mogłaby znacząco przewyższać oferty sprzedaży, co zniekształciłoby symulację rynku. Limity te służą zachowaniu równowagi w testach, uwzględniając, że sprzedaż akcji wymaga ich wcześniejszego posiadania. Dane o firmach są uporządkowane według id firmy. W takiej sytuacji po wprowadzeniu limitów zapytań dla jednego cyklu, firmy znajdujące się w pierwszej kolejności na liście dostałby więcej ofert. Przemieszanie listy, które następuje po otrzymaniu danych pozwala uniknąć tego problemu.



Rys. 3.16 Schemat zachowania aktywnego użytkownika z analizą rynku

Na Rys. 3.17 przedstawiony został kod klasy użytkownika przeglądającego. Klasa ta została zaimplementowana przy użyciu frameworka Locust, który umożliwia symulowanie zachowań użytkowników korzystających z aplikacji w celu obciążeniowego testowania jej wydajności.

Na początku klasy WebsiteReadOnlyUser zdefiniowana jest waga użytkownika, która określa częstość jego pojawiania się w symulacjach w stosunku do innych klas użytkowników. Kolejnym kluczowym elementem jest wait_time, która w tej klasie korzysta z funkcji constant(). Parametr ten odpowiada za stałą ilość czasu, jaką użytkownik czeka między kolejnymi akcjami, co symuluje realne użytkowanie aplikacji w równych odstępach czasowych.

W metodzie on_start() zdefiniowany jest proces rejestracji użytkownika oraz tworzenia firmy. Ta metoda jest wywoływana raz na początku testu. Po wygenerowaniu losowych danych logowania za pomocą funkcji generate_random_data(), użytkownik rejestruje się poprzez endpoint /api/auth/signUp. Następnie inicjuje proces logowania i zostaje przydzielony mu token autoryzacyjny. Token ten jest kluczowy, ponieważ przy każdym kolejnym żądaniu do API będzie on wykorzystywany do uwierzytelnienia użytkownika. Ostatni krok w procesie inicjacji to tworzenie firmy przez użytkownika. Realizacja tego zadania odbywa się poprzez endpoint /api/company/addCompany. Cały proces został zsynchronizowany za pomocą bariery all_locusts_spawned.wait(), która czeka na inicjalizację wszystkich użytkowników testowych, zanim rozpoczną wykonywanie swoich zadań. Funkcja on_start() dla innych klas jest dokładnie taka sama.

Najważniejszą częścią klasy jest metoda oznaczona jako zadanie Locusta – @task. W tym przypadku przedstawiono metodę getSellOffers(), która symuluje pobieranie ofert sprzedaży akcji przez użytkownika. Każda z metod gracza przeglądającego jest implementowana w analogiczny sposób, a jedyną różnicą między nimi są endpointy, z których korzystają. Podejście do implementacji zadań aktywnego użytkownika oraz użytkownika z analizą rynku przebiega w podobny sposób. Dla tych klas konieczne było dodanie warunków zgodnie z ich schematami działania.

```
52  class WebsiteReadOnlyUser(FastHttpUser):
53      weight = 1
54      wait_time = constant(time)
55      token = ''
56
57      def on_start(self):
58          login = generate_random_data()
59          self.client.post("/api/auth/signUp", json=login)
60          data = {
61              'username': login['username'],
62              'password': login['password']
63          }
64          response = self.client.post("/api/auth/signIn",json=data)
65          self.token = response.json()['token']
66          company_name = fake.company()
67          self.client.post("/api/company/addCompany",headers={"authorization": "Token " + self.token}, json={"name":company_name})
68          all_locusts_spawned.wait()
69
70      @task
71      def getSellOffers(self):
72          self.client.get("/api/user/sellOffers", headers={"authorization": "Token " + self.token})
```

Rys. 3.17 Fragment pliku loctusfile.py z klasą użytkownika przeglądającego

3.2.3 Pomiar wydajności w testach

Podczas testów wydajnościowych kluczowym aspektem było monitorowanie parametrów systemu, które odzwierciedlają jego zachowanie pod obciążeniem. Zdecydowano się na zapisywanie czasów odpowiedzi API, czasu przetwarzania na bazie danych oraz czas wykonywania aplikacji. Mierzone były również parametry związane z wykorzystaniem zasobów systemowych, takich jak zużycie procesora i pamięci

operacyjnej RAM. Pomiar tych danych dostarcza szczegółowych informacji na temat obciążenia systemu, pozwalając na identyfikację ewentualnych wąskich gardeł oraz potencjalnych problemów wydajnościowych.

Poniżej na Rys. 3.18 przedstawiono sposób pomiaru i zapisu zużycia CPU oraz pamięci RAM dla poszczególnych kontenerów. Kod odpowiedzialny za te pomiary regularnie monitoruje zasoby przydzielone dla kontenerów, zapisując wyniki do bazy danych, co pozwala na ich późniejszą analizę i korelację z innymi danymi testowymi.

Dla każdego kontenera uruchamiany jest osobny wątek, tak aby zbierać dane równolegle. Rozwiązanie to jest specyficzne dla Dockera. Pomiary zużycia zasobów są wykonywane cyklicznie co dwie sekundy za pomocą funkcji log_container_usage(container). W pierwszej kolejności odczytywany jest bieżący czas oraz aktualne statystyki dotyczące przekazanego w argumencie kontenera. Z poziomu tych statystyk obliczane jest zużycie procesora w procentach oraz zużycie pamięci w megabajtach. Otrzymane wartości są następnie zapisywane w bazie danych do tabeli cpu, gdzie każdemu wpisowi przypisywany jest unikalny identyfikator kontenera, dla którego dokonano pomiaru.

```
32  def log_container_usage(container):
33      while True:
34          timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
35          stats = container.stats(stream=False)
36          cpu_percentage = calculate_cpu_percentage(stats)
37          memory_usage = stats['memory_stats']['usage'] / (1024 * 1024)
38          sql_insert = """
39          INSERT INTO "stockApp_cpu" (timestamp, cpu_usage, memory_usage, contener_id)
40          VALUES (%s, %s, %s, %s)
41          """
42          cursor.execute(sql_insert, (timestamp, cpu_percentage, memory_usage, container.name))
43          conn.commit()
44          time.sleep(2)
```

Rys. 3.18 Pomiar zużycia procesora i pamięci RAM

Zgodnie z założeniami konieczne było dodanie pomiaru czasu każdego endpointa w aplikacji giełdy. Idealnym narzędziem do implementacji tej funkcjonalności, dla czasu aplikacji i bazy danych w aplikacji giełdy, okazał się middleware. Czyli dodatkowa warstwa wykonująca się przy każdym zapytaniu. Na Rys. 3.19 znajduje się fragment kodu wykonywany tuż przed wysłaniem odpowiedzi. Najpierw w linii 14 sprawdzamy czy response ma atrybut data, czyli czy zwracane są dane. Robimy to, aby mierzyć czas tylko dla zapytań, które zakończyły się sukcesem. Kolejny krok, realizowany w linii 15 oraz 16 to obliczenie czasu aplikacji i bazy danych. Następnie do odpowiedzi dołączamy unikalny request_id. Pierwotnie zakładano dodanie tego parametru do nagłówka odpowiedzi, aby uniknąć sprawdzania czy odpowiedź to lista czy słownik. Okazało się, że framework Locust stosowany w obciążniku nie ma dostępu do nagłówków i koniczne było przekazanie

informacji w ciele odpowiedzi. Ostatnim krokiem było dodanie pomiarów do bazy danych do tabeli marketlog, aby potem mogły być wykorzystane w analizie.

```
13      def process_response(self, request, response):
14          if hasattr(request, 'start_time') and hasattr(response, 'data'):
15              total_time = time.time() - request.start_time
16              db_time = sum(float(query['time']) for query in connection.queries)
17              id = None
18              if isinstance(response.data, list) and 'request_id' in response.data[-1]:
19                  id = response.data[-1]["request_id"]
20              if isinstance(response.data, dict) and 'request_id' in response.data:
21                  id = response.data['request_id']
22              if id:
23                  MarketLog.objects.using('test').create(
24                      api_method=request.method,
25                      application_time=total_time,
26                      database_time=db_time,
27                      endpoint_url=request.get_full_path(),
28                      request_id=id,
29                      timestamp = datetime.datetime.now()
30                  )
31          return response
```

Rys. 3.19 Fragment kodu LogRequestMiddleware.py

Kolejną istotną rzeczą było dodanie pomiaru czasu aplikacji oraz bazy danych w trakcie trwania transakcji. W Celery nie udało się zaaplikować middleware, tak jak to oferowało Django dla pomiaru zapytań API. Konieczne było napisanie kodu od zera. Z pomiarem czasu aplikacji nie ma większych problemów bo po prostu mierzymy czas na samym początku funkcji oraz na samym końcu i obliczamy różnicę. Na Rys. 3.20 dokładnie widać w jaki sposób został zmierzony czas aplikacji. Do pomiaru użyto funkcji time.time(), która w zależności od sytemu może mieć dokładność pomiędzy jedną mikro sekundą, a jedną mili sekundą.

```
13  def executeTransactions(companyIds):
14      try:
15          startTime = time.time()
16  >       for companyId in companyIds: ···
94  >       if numberOfSellOffers == 0 or numberOfBuyOffers == 0:
96          endTime = time.time()
97          applicationTime = endTime - startTime
```

Rys. 3.20 Fragment kodu tasks.py pomiar czasu aplikacji

Bardziej złożony jest jednak pomiar czasu związanego z operacjami bazy danych, który musi zostać wykonany osobno dla każdej operacji wysyłającej zapytanie do bazy. Jeden z takich pomiarów został pokazany na Rys. 3.21. Każdy blok kodu, który zawiera zapytania do bazy danych, w tym przypadku zapytania dotyczące ofert kupna i sprzedaży,

jest otoczony pomiarem czasu przed i po jego wykonaniu. Czas trwania zapytań jest następnie sumowany do zmiennej database_time, która przechowuje łączny czas spędzony na operacjach bazodanowych w ramach całego procesu.

```
20            db_start_time = time.time()
21            buy_offers = BuyOffer.objects.filter(company_id=company_id, actual=True).order_by('-maxPrice')
22            sell_offers = SellOffer.objects.filter(company_id=company_id, actual=True).order_by('minPrice')
23            db_end_time = time.time()
24            database_time += db_end_time - db_start_time
```

Rys. 3.21 Fragment kodu tasks.py z pomiarem czasu bazy danych

Na koniec procesu następuje zebranie wszystkich zgromadzonych danych i zapisanie ich do tabeli tradelog w bazie danych, co umożliwia ich późniejszą analizę i porównanie w kontekście testów wydajnościowych. Ten proces widoczny jest na Rys. 3.22.

```
102                                 db_end_time = time.time()
103                                 database_time += db_end_time - db_start_time
104                             else:
105                                 continue
106         if number_of_sell_offers == 0 and number_of_buy_offers == 0:
107             return company_ids
108         end_time = time.time()
109         application_time = end_time - start_time
110
111         TradeLog.objects.using('test').create(
112             application_time=application_time,
113             database_time=database_time,
114             number_of_sell_offers=number_of_sell_offers,
115             number_of_buy_offers=number_of_buy_offers,
116             timestamp = datetime.datetime.now(),
117             company_ids = company_ids
118             )
119         return company_ids
```

Rys. 3.22 Fragment kodu tasks.py z zapisem do bazy danych

Ostatnim agregowanym czasem jest czas odpowiedzi API. Na ten czas składają się opisane w tym podrozdziale czas aplikacji giełdy oraz czas bazy danych. Do obsłużenia zapisu danych do tabeli tradelog użyto, oferowanego przez Locust, zdarzenia @events.request.add_listener, które jest wywoływane za każdym razem, gdy dostarczona jest odpowiedź na zapytanie. Szczegóły funkcji widać na Rys. 3.23. W pierwszej kolejności pobrano wartość pola request_id. Jest to niezbędne, aby było wiadome, którego zapytania dotyczyła odpowiedź. Jeżeli w odpowiedzi nie ma pola request_id to oznacza, że zapytanie nie zakończyło się sukcesem. Ustalono że dane z takich zapytań nie będą zapisywane do bazy danych. Następnie zostaje pobrany aktualny czas w odpowiednim formacie. Na koniec pozostaje zapisać czas odpowiedzi wraz ze znacznikiem czasowym oraz request_id do tabeli trafficlog. Uznano że czas API najlepiej zapisać w jednostkach SI czyli sekundach.

```
217    @events.request.add_listener
218    def log_request(request_type, name, response_time, response_length, response, context, exception, **kwargs):
219        try:
220            if isinstance(response.json(), list):
221                id = response.json()[-1]["request_id"]
222            else:
223                id = response.json()['request_id']
224        except (ValueError, KeyError):
225            return
226        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
227        cursor.execute(
228            """INSERT INTO "stockApp_trafficlog" (timestamp, request_id, api_time) VALUES (%s, %s, %s)""",
229            (timestamp, id, response_time / 1000.0)
230        )
231        conn.commit()
```

Rys. 3.23 Fragment kodu locustfile.py z zapisem danych do tabeli trafficlog

4 Eksperymenty

W ramach przeprowadzonych eksperymentów przetestowano funkcjonowanie systemu giełdowego pod różnym obciążeniem. Celem badań było zrozumienie, w jaki sposób różne parametry wpływają na wydajność systemu oraz jego stabilność. Wyniki testów zostały poddane szczegółowej analizie.

4.1 Metodyka badań

Celem przeprowadzonych badań jest zrozumienie zachowań systemu z różnymi parametrami, a następnie identyfikacja kluczowych zależności między uzyskanymi danymi, takimi jak zużycie zasobów, czasy odpowiedzi, czasy aplikacji oraz czasy bazy danych. Zakres badań obejmuje zarówno analizy podstawowe, jak i bardziej zaawansowane metody analityczne, takie jak reguły asocjacyjne oraz klasyfikacje danych.

4.1.1 Badania podstawowe

Badania podstawowe miały na celu wykrycie zależności, które można było zauważyć na wykresach bez zastosowana specjalistycznych metod. Wnioski wyciągnięte na tym etapie były pomocne w kontekście tego, na jakie elementy systemu należy zwrócić uwagę w dalszych badaniach.

Badano następujące zależności:

1) Średni czas odpowiedzi z podziałem na punkty końcowe. Osobne zestawienie dla każdego testu.
2) Średni czas trwania transakcji dla wszystkich firm w jednym cyklu.
3) Czas odpowiedzi w trakcie trwania testu.
4) Czas transakcji dla każdej grupy firm w trakcie trwania testu.
5) Zużycie procesora.
6) Zużycie pamięci RAM.

Zależności od punktu drugiego do szóstego miały zebranie wyniki na jednym wykresie dla testów z tym samym zmienianym parametrem. W punkcie piątym i szóstym zastosowano podział na kontener, którego dotyczyło zużycie danego zasobu.

4.1.2 Badania z wykorzystaniem reguł asocjacyjnych [11]

Sposób zapisu reguł asocjacyjnych jest przedstawiony we wzorze (4.1).

$$A \Rightarrow B \tag{4.1}$$

gdzie: A – zbiór poprzedników, B – zbiór następników.

Kluczowe miary opisujące reguły asocjacyjne to:

1) Support (wsparcie) – określa jak często występuje dana reguła. Obliczane jest zgodnie ze wzorem (4.2).

$$Support(A \Rightarrow B) = \frac{|A \cap B|}{|\Omega|} \tag{4.2}$$

gdzie: A – zbiór poprzedników, B – zbiór następników ,Ω - cały badany zbiór.

2) Confidence (ufność) – miara tego, jak często występuje reguła w stosunku do liczebności poprzednika. Do obliczeń stosowany jest wzór (4.3)

$$Confidence(A \Rightarrow B) = \frac{|A \cap B|}{|A|} \tag{4.3}$$

3) Lift (podniesienie) - miara, która określa siłę zależności między dwoma elementami lub zbiorami elementów. Podniesienie większe od jeden oznacza, że wystąpienie A zwiększa prawdopodobieństwo wystąpienia B. Dla lift mniejszego od jeden wystąpienie A oznacza mniejsze prawdopodobieństwo na wystąpienie B. Wartość jeden dla podniesienia oznacza, że nie ma zależności pomiędzy A i B. Poniesienie wyrażane jest za pomocą wzoru (4.4).

$$Lift(A \Rightarrow B) = \frac{Support(A \Rightarrow B)}{Support(A) * Support(B)} \tag{4.4}$$

gdzie: A – zbiór poprzedników, B – zbiór następników.

W kontekście badanego systemu zastosowano reguły asocjacyjne do ustalenia, które parametry współwystępują. Do przeprowadzenia analizy konieczna była zmiana danych na binarne. Gdy dana wartość była większa od średniej to przyjmowała wartość jeden, w innym wypadku jej wartość to zero. Testowano zmianę na dane binarne na podstawie mediany oraz kilku stałych wartości. Nie zauważono większych różnic w uzyskanych wynikach, dlatego zdecydowano się pozostać przy wartości średniej jako kryterium podziału na dane binarne.

Zmieniono następujące parametry:

1) Czas aplikacji giełdy
2) Czas bazy danych giełdy
3) Czas API giełdy
4) Czas aplikacji transakcji
5) Czas bazy danych transakcji
6) Zużycie procesora aplikacji giełdy
7) Zużycie procesora bazy danych giełdy
8) Zużycie procesora bazy danych testu
9) Zużycie procesora kontenera odpowiedzialnego za transakcje

4.1.3 Badania z wykorzystaniem klasyfikatorów

Klasyfikacja jest jednym z podstawowych zadań w dziedzinie uczenia maszynowego i polega na przypisywaniu obiektom etykiet z określonego zbioru klas na podstawie ich cech. Proces ten można rozumieć jako próbę rozpoznania wzorców w danych, które następnie pozwalają na określenie przynależności obiektu do jednej z ustalonych kategorii. W przypadku problemów klasyfikacyjnych algorytm jest trenowany na zbiorze danych zawierającym przykłady opatrzone poprawnymi etykietami. Na podstawie tych oznaczonych przykładów model uczy się, jakie właściwości (cechy) odróżniają poszczególne klasy, co pozwala mu w przyszłości klasyfikować nowe, nieoznaczone wcześniej obiekty [12].

Ocena jakości modelu klasyfikacyjnego jest kluczowym etapem procesu budowy modeli uczenia maszynowego. W tym celu stosuje się różnorodne metryki, które pozwalają na kompleksową ocenę skuteczności algorytmu. Zwykle można obliczyć metryki na dwa sposoby jako makrośrednia oraz jako mikrośrednia. Makrośrednia daje równą wagę każdej klasie i jest bardziej odpowiednia, gdy chcemy ocenić efektywność modelu dla każdej klasy indywidualnie, niezależnie od ich liczebności. Mikrośrednia jest bardziej przydatna, gdy interesuje nas globalna wydajność modelu, zwłaszcza gdy mamy niezrównoważony zbiór danych [13]:

1) Accuracy (dokładność) – jest to odsetek poprawnie sklasyfikowanych próbek w całym zbiorze danych testowych. Jest to najbardziej intuicyjna metryka, jednak może być myląca w przypadku niezrównoważonych danych, gdzie niektóre klasy są reprezentowane znacznie liczniej niż inne. W takich przypadkach model może uzyskać wysoką dokładność, przewidując większość próbek jako dominującą klasę,

mimo że nie radzi sobie z rzadziej występującymi klasami. Obliczana według wzoru (4.5).

$$\text{Accuracy} = \frac{\sum_{i=1}^{N} TP_i + TN_i}{\sum_{i=1}^{N} TP_i + TN_i + FP_i + FN_i} \tag{4.5}$$

gdzie: N – liczba klas, TP_i – prawdziwy pozytyw dla i-tej klasy, TN_i – prawdziwy negatyw dla i-tej klasy, FP_i – fałszywy pozytyw dla i-tej klasy, FN_i – fałszywy negatyw dla i-tej klasy.

2) Precision (precyzja) – jest to stosunek liczby poprawnie sklasyfikowanych próbek danej klasy do całkowitej liczby próbek przypisanych do tej klasy. Wysoka precyzja oznacza, że model rzadko przypisuje daną klasę błędnie, co jest szczególnie ważne w zastosowaniach, gdzie błędna klasyfikacja może mieć poważne konsekwencje. Wzór (4.6) opisuje sposób makro, a (4.7) mikro obliczania tej metryki.

$$\text{Precision}_{macro} = \frac{1}{N} * \sum_{i=1}^{N} \frac{TP_i}{TP_i + FP_i} \tag{4.6}$$

$$\text{Precision}_{micro} = \frac{\sum_{i=1}^{N} TP_i}{\sum_{i=1}^{N} TP_i + \sum_{i=1}^{N} FP_i} \tag{4.7}$$

gdzie: oznaczenia jak w równaniu (4.5)

3) Recall (czułość) – znana także jako swoistość, odnosi się do stosunku liczby poprawnie sklasyfikowanych próbek danej klasy do całkowitej liczby próbek, które rzeczywiście należą do tej klasy. Wysoka czułość oznacza, że model skutecznie wykrywa wszystkie przypadki danej klasy, co jest istotne w zastosowaniach, gdzie kluczowe jest uniknięcie pominięcia danej klasy (np. w detekcji oszustw). Do obliczeń makro stosowany jest wzór (4.8), natomiast dla mikro (4.9).

$$Recall_{macro} = \frac{1}{N} * \sum_{i=1}^{N} \frac{TP_i}{TP_i + FN_i} \tag{4.8}$$

$$Recall_{micro} = \frac{\sum_{i=1}^{N} TP_i}{\sum_{i=1}^{N} TP_i + \sum_{i=1}^{N} FN_i} \tag{4.9}$$

gdzie: oznaczenia jak w równaniu (4.5)

4) F1-score (Wartość F1) – jest to średnia harmoniczna precyzji i czułości, która pozwala na zrównoważenie tych dwóch metryk. Wartość F1 jest szczególnie użyteczna, gdy mamy do czynienia z niezrównoważonymi danymi, ponieważ

wysoka wartość tej metryki oznacza zarówno wysoką precyzję, jak i czułość. Marko obliczany zgodnie ze wzorem (4.10), a mikro zgodnie z (4.12).

$$F1_{macro} = \frac{1}{N} * \sum_{i=1}^{N} F1_i \tag{4.10}$$

gdzie: N – liczba klas, F_i – obliczamy według wzoru(4.11)

$$F1 = \frac{2 * \text{Recall} * \text{Precision}}{Recall + \text{Precision}} \tag{4.11}$$

$$F1_{micro} = \frac{2 * Recall_{micro} * \text{Precision}_{micro}}{Recall_{micro} + \text{Precision}_{micro}} \tag{4.12}$$

5) Macierz pomyłek, zwana również macierzą konfuzji, jest narzędziem służącym do wizualizacji wyników klasyfikacji. Pozwala ona na ocenę liczby poprawnych oraz błędnych klasyfikacji, umożliwiając bardziej szczegółową analizę działania modelu. Dla problemu binarnego macierz pomyłek ma postać przedstawioną na równaniu (4.13).

$$\begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix} \tag{4.13}$$

gdzie: TP – prawdziwy pozytyw, TN – prawdziwy negatyw, FP – fałszywy pozytyw, FN – fałszywy negatyw.

W badanym systemie klasyfikacje wykorzystano do przewidywania jaka klasa użytkownika wykonała zapytanie API.

4.2 Testy

W początkowej fazie przeprowadzono szereg testów z krótkim limitem czasowym, aby zobaczyć jakie parametry powodują zmiany w systemie. Ustalono, że w dalszej analizie pod uwagę będzie brana liczba użytkowników, czas pomiędzy transakcjami, czas pomiędzy zapytaniami, liczba kontenerów odpowiedzialnych za transakcje oraz klasy użytkowników biorących udział w symulacji.

Z kolei parametry stałe dla każdego testu to:

1) Czas testu wynoszący godzinę.
2) Ramp up, czyli czas pojawiania się nowych użytkowników na początku testy, równy jeden nowy użytkownik na sekundę. Podniesienie tego parametru skutkowało dużym obciążeniem już w momencie startu testu. Założono łagodny start testów, gdy każdy

użytkownik zakłada konto i się loguje, z uwagi na to, że te działania były bardziej wymagające w porównaniu do innych zadań.

3) Czas pomiędzy kolejnymi aktualizacjami cen akcji firm ustalony na 15 sekund. Nie zauważono wpływu tego czynnika na inne części systemu.
4) Czas pomiędzy następną zmianą stanu pieniężnego użytkowników ustalony na pięć sekund. Wymagane jest aby ta akcja była wywoływana jak najczęściej.
5) Czas aktualizacji statusu przeterminowanych ofert. Uznano, że to działanie nie powinno być częste i było realizowane co 60 sekund.

Tabela 4.1 przedstawia zestawienie poszczególnych parametrów dla każdego testu. Testy można podzielić na pięć grup: G1={Test 1, Test 2, Test 3, Test 4}; G2={Test 5, Test 6, Test 7, Test 8}; G3={Test 9, Test 10, Test 11, Test 12}; G4={Test 13, Test 14, Test 15, Test 16}; G5={Test 17, Test 18, Test 19, Test 20}. W G1 zmieniono liczbę użytkowników. G2 charakteryzuje się zmianą czasów między zapytaniami. W G3 zmienia się czas pomiędzy transakcjami. G4 to grupa, w której zmienną jest liczba kontenerów wykonujących transakcje. G5 zawiera testy ze zmieniającą się liczbą użytkowników poszczególnych klas przy ogólnej stałej liczbie użytkowników.

Tabela 4.1 Opis wykonanych testów

Nr testu	A	NA	AZA	Ttr[s]	Trq[ms]	Ktr
Test 1	10	0	0	30	600	3
Test 2	50	0	0	30	600	3
Test 3	90	0	0	30	600	3
Test 4	130	0	0	30	600	3
Test 5	60	0	0	30	200	3
Test 6	60	0	0	30	500	3
Test 7	60	0	0	30	800	3
Test 8	60	0	0	30	1100	3
Test 9	60	0	0	5	600	3
Test 10	60	0	0	20	600	3
Test 11	60	0	0	35	600	3
Test 12	60	0	0	50	600	3
Test 13	60	0	0	30	600	2
Test 14	60	0	0	30	600	4
Test 15	60	0	0	30	600	6
Test 16	60	0	0	30	600	8
Test 17	60	0	0	30	600	3
Test 18	24	12	24	30	600	3
Test 19	40	20	0	30	600	3
Test 20	30	0	30	30	600	3

Nazwy kolumn oznaczają:

1) A – liczba aktywnych użytkowników.
2) NA – liczba użytkowników przeglądających(nieaktywnych).

3) AZA – liczba użytkowników z analizą trendów cen.
4) Ttr – czas pomiędzy transakcjami.
5) Trq – czas pomiędzy kolejnymi zapytaniami użytkowników.
6) Ktr – liczba kontenerów odpowiedzialnych za przeprowadzanie transakcji.

Do analizy zostały wykorzystane następujące biblioteki:

1) Pandas – użyta do przetwarzania plików .csv [14].
2) Matplotlib – służy do rysowania wykresów i wizualizacji danych [15].
3) Scikit-learn – biblioteka zawierająca narzędzia używane w uczeniu maszynowym [16].
4) Mlxtend – kolejna biblioteka wykorzystywana w uczeniu maszynowym. Wykorzystano ją do znalezienia reguł asocjacyjnych [17].
5) Xgboost – biblioteka implementująca algorytm Extreme Gradient Boosting. Użyty do klasyfikacji danych [18].

Po przeprowadzeniu testów przygotowano skrypty w języku Python57[19]:

1) prepareData.ipynb – Ten skrypt miał za zadanie przegotować dane do dalszej analizy. Realizuje następujące zadania:
   -konwersja danych z plików .sql do plików .csv,
   -sortowanie danych względem kolumny timestamp
   -połączenie tradelog.csv z trafficlog.csv, aby otrzymać kompletne logi dotyczące zapytań API,
   -wydzielanie osobnych plików dla zużycia zasobów dla każdego kontenera,
   -dokładne określenie przedziałów czasowych, w których są przeprowadzane transakcje,
   -połączenie wyżej wymienionych danych do jednego pliku, który następnie został wykorzystany w regułach asocjacyjnych oraz klasyfikacji.

2) dataAnalysis.ipynb – realizuje zadania opisane w zakresie Badania podstawowe.
3) associationRules.ipynb – wykonywane są czynności opisane w zakresie Badania z wykorzystaniem reguł asocjacyjnych.
4) classification.ipynb – skrypt odpowiedzialny za realizację zadań opisanych w zakresie Badania z wykorzystaniem klasyfikatorów.

W tytułach lub etykietach wykresów są parametry testu. Na przykład 60_A_30s_600ms_3. Pierwsza liczba oznacza liczbę użytkowników. Druga klasę użytkowników. Trzecia liczba to czas pomiędzy transakcjami. Czwarta to czas pomiędzy kolejnymi zapytaniami użytkownika. Ostatnia to liczba kontenerów odpowiedzianych za realizację transakcji.

Na wykresach przedstawiających czas API w trakcie całego testu w celu zwiększenia czytelności wzięto średnią z pięciu sekund. Natomiast dla zużycia procesora brano średnią z 15s. Czas z endpointów /api/auth/signUp, /api/auth/signIn oraz api/company/addCompany nie był brany pod uwagę przez to, że w każdym teście czynności te były wykonywane tylko na początku na nieobciążonym systemie i zawsze dawały ten sam wynik.

4.3 Analiza

Pierwszą obserwacją jaką zauważono było to, zużycie pamięci Ram nie było kluczowe dla wydajności i nie zaobserwowano żadnych znaczących różnic. Zauważono, że zużycie procesora w kontenerach aplikacji giełdy, bazy danych giełdy, bazy danych testów oraz workera realizującego transakcje jest najbardziej istotne dla wydajności systemu.

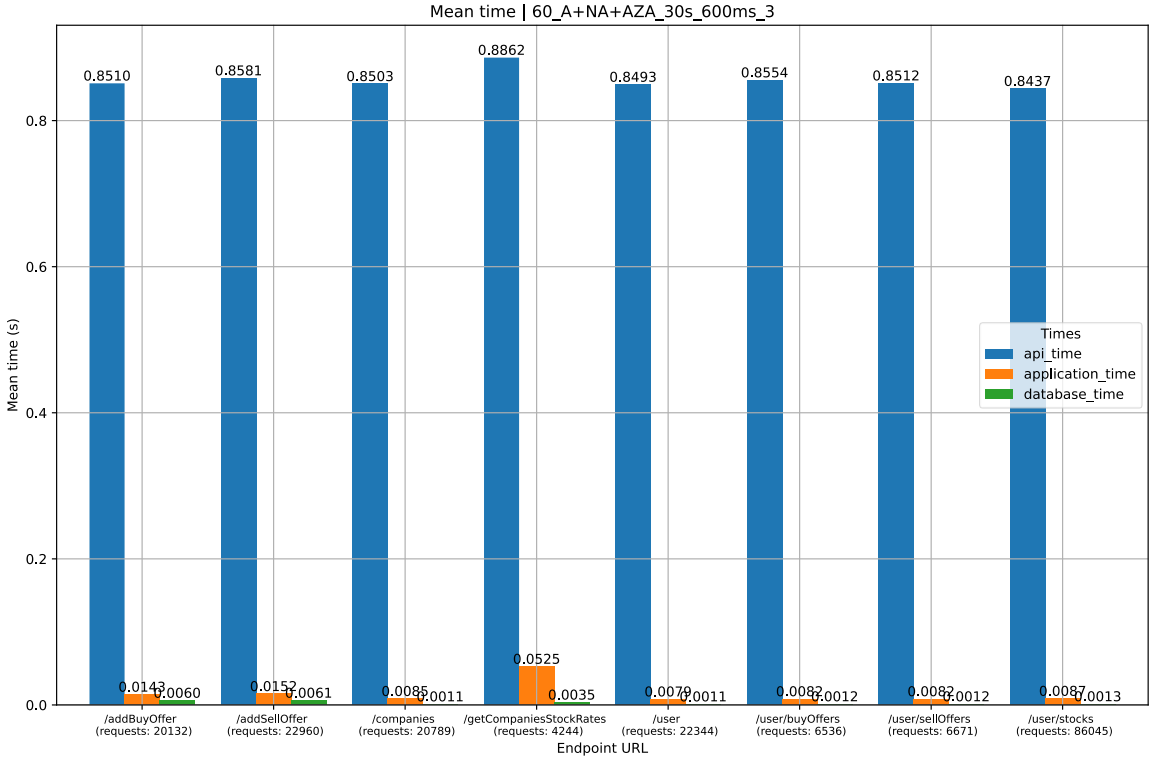
Następnie analiza wykazała, że czas bazy danych dla poszczególnych punktów końcowych był taki sam niezależnie od parametrów. Podobna zależność jest widoczna dla czasu aplikacji, choć ten parametr wykazywał zmiany na poziomie jednej tysięcznej sekundy, co uznano za nieznaczącą zmianę. Na Rys. 4.1 zaprezentowano czasy zapytań dla testu 18, w którym był każdy rodzaj użytkowników i można było zaobserwować działanie wszystkich używanych endpointów.

Dla /companies, /user, /user/stocks, /user/buyOffer, /user/sellOfer czas bazy danych był równy około 0.001s, a czas aplikacji był z przedziału <0.0079s;0.009s.>. Czas API tych punktów końcowych był zbliżony. Różnice w czasie API były na poziomie kilku dziesięciotysięcznej sekundy. Wśród tej grupy na przestrzeni wszystkich testów nie zaobserwowano, by ich czasy API dało się posortować w tej samej kolejności

Dla punktów końcowych dodających ofertę kupna lub sprzedaży czas bazy danych był równy 0.06s. Dodanie oferty sprzedaży wiązało się czasem aplikacji równym około 0.0152s i był on o 0.001s wyższy od czasu aplikacji oferty kupna. Czas API dla ofert sprzedaży był większy o około 0.007s.

Najbardziej obciążający był endpoint /getCompaniesStockRates. Jego czas bazy danych był równy 0.0035s, czyli około dwa razy mniejszy niż ten dla dodawania ofert. Natomiast jego czas aplikacji równy 0.0525s był zdecydowanie najdłuższy.

Jak widać na Rys. 4.1 różnice w czasie API są marginalne dla wszystkich punktów końcowych poza /getCompaniesStockRates, który ma większy czas aplikacji. W dalszej części analizy czas bazy danych i czas aplikacji nie był brany pod uwagę. Podział na poszczególne endpointy również uznano za zbędny.



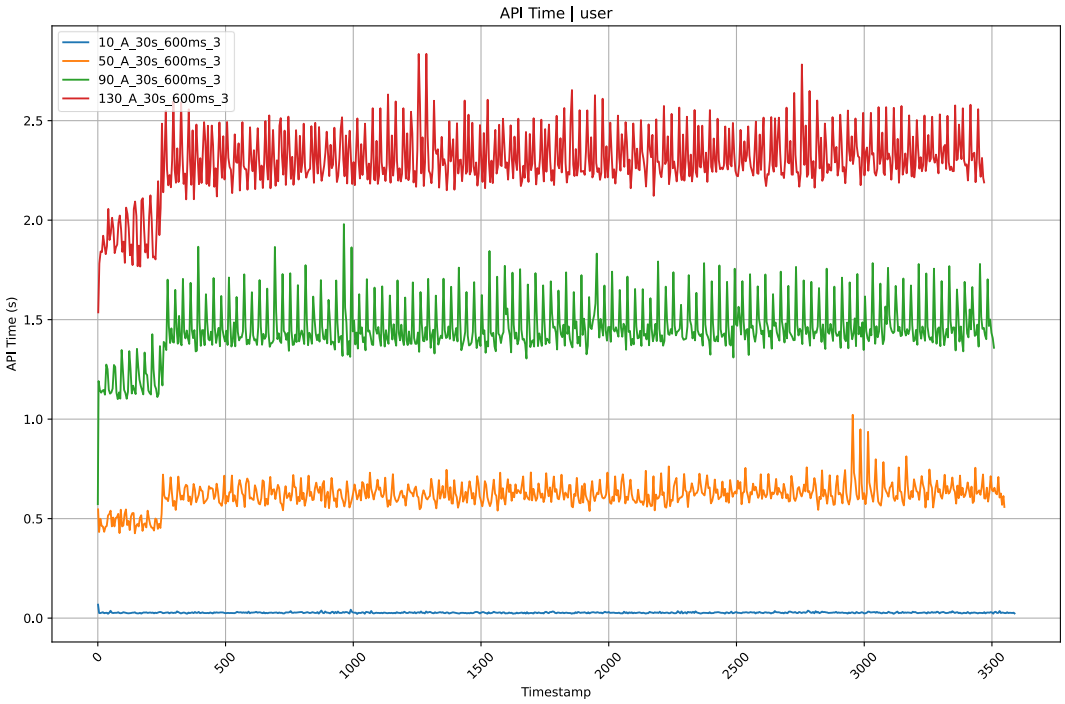
Rys. 4.1 Średnie czasy dla testu 18

Gdy zmienianym parametrem była liczba użytkowników to zauważono zależność, z której wynika, że im więcej użytkowników tym dłuższy czas odpowiedzi API. Przedstawiono to na

Rys. 4.2. Okresowe skoki najlepiej widoczne dla 90 użytkowników są spowodowane tym że przeprowadzane są transakcje.

Tabela 4.2 przedstawia średnie zużycie procesora dla wybranych kontenerów. Jak widać aplikacja giełdy, generator zapytań, i obie bazy danych miały dwa razy mniejsze obciążenie, gdy liczba użytkowników była równa 10. W pozostałych przypadkach zużycie zasobów jest na podobnym poziomie. Kontener realizujący transakcje dla 10 graczy generował obciążenie mniejsze o około 33% od reszty. W innych rodzajach testów wykorzystanie procesora jest bardzo podobne jak dla testu drugiego, trzeciego i czwartego. Różnice pojawiły dla pojedynczych kontenerów i gdy wystąpiły zostały niżej szczegółowo omówione.

Przy różnej liczbie użytkowników w teście występuje różna liczba firm. Przy wykonywaniu transakcji następuje podział firm na grupy. Liczba tych grup zależy od liczby firm, dlatego dla testów z różną liczbą użytkowników nie przeprowadzono analizy co do długości trwania transakcji, dlatego że każdy test miał inną liczbę firm i uznano, że w takiej sytuacji wyciągnięte wnioski, co do czasu wykonywania transakcji mogłyby być nieprawdziwe.

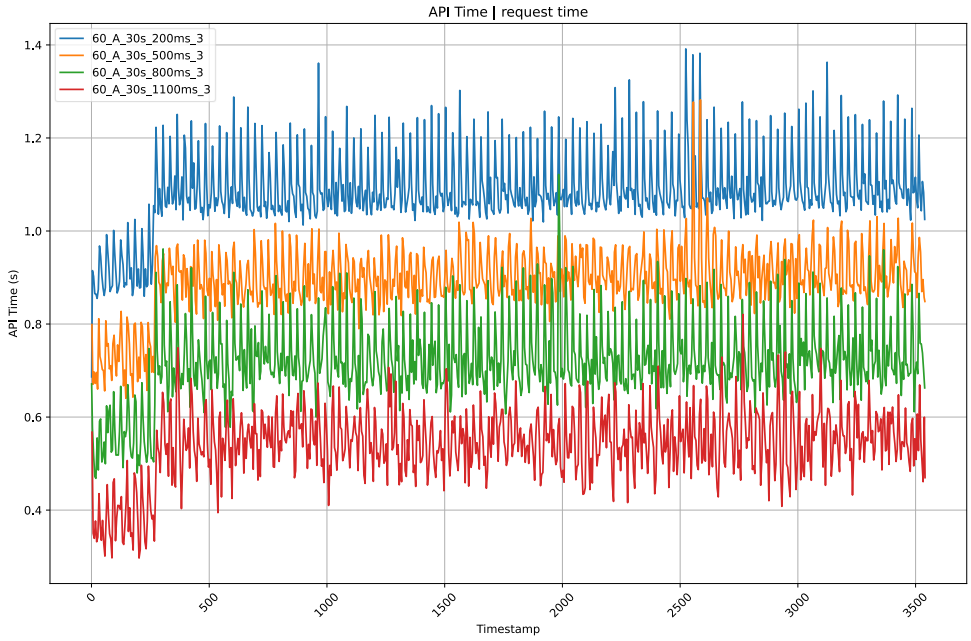


Rys. 4.2 Czas API w trakcie trwania testu –[test 1, test 2,test 3, test 4]

Tabela 4.2 Średnie zużycie procesora [test 1, test 2, test 3, test 4]

	Aplikacja giełdy	Aplikacja obciążająca	Baza danych giełdy	Baza danych testu	Kontener realizujący transakcje
Test 1	19.95%	4.13%	17.33%	11.92%	31.45%
Test 2	39.53%	7.34%	38.51%	24.94%	43.43%
Test 3	39.55%	7.41%	40.45%	24.41%	48.51%
Test 4	39.37%	7.51%	38.30%	24.68%	44.41%

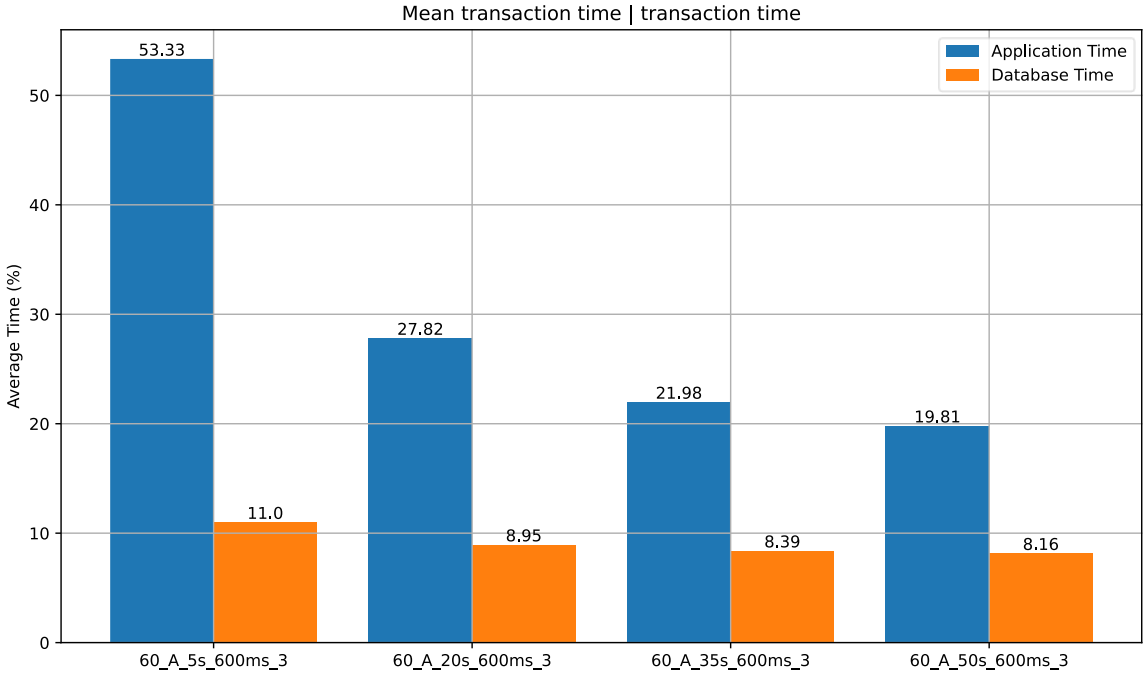
Przy zmianie czasu pomiędzy kolejnymi zapytaniami graczy zmieniał się czas API. Na Rys. 4.3 zaprezentowano jak ten czas kształtował się w trakcie trwania testów. Można zauważyć że czas oczekiwania na odpowiedź rósł wraz ze spadkiem czasu pomiędzy zapytaniami.



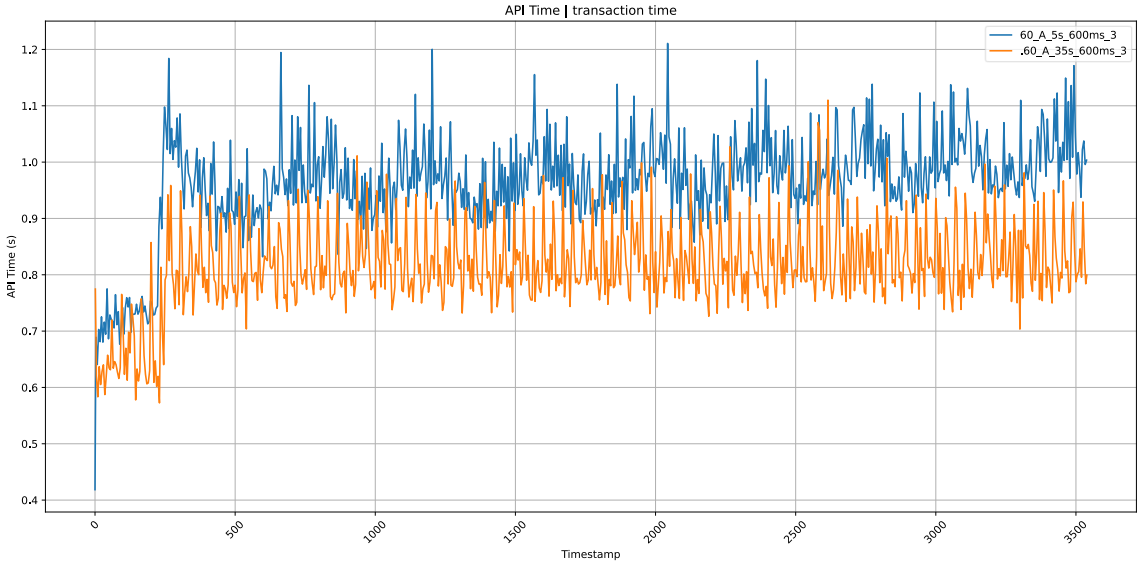
Rys. 4.3 Czas API trakcie trwania testu – [test 5, test 6, test 7, test 8]

Następna grupa testów to ta, w której zmienianym parametrem był czas pomiędzy realizacją kolejnych transakcji. Na Rys. 4.4 przedstawiono porównanie średniego czasu realizacji transakcji (w procentach) przez aplikację oraz bazę danych, w zależności od czasu pomiędzy transakcjami. Wyniki jednoznacznie wskazują, że im krótszy czas pomiędzy transakcjami, tym większy procent tego czasu zajmuje sama realizacja transakcji. Dla czasu równego pięć sekund, aplikacja zużywała 53,33% czasu, natomiast dla dłuższych czasów, takich jak 35 czy 50 sekund, zauważalny był spadek tego udziału do wartości odpowiednio 21,98% i 19,81%. Z drugiej strony, czas obsługi przez bazę danych utrzymywał się na stabilnym poziomie, co wskazuje na mniejszy wpływ zmiennego interwału między transakcjami na tę część systemu.

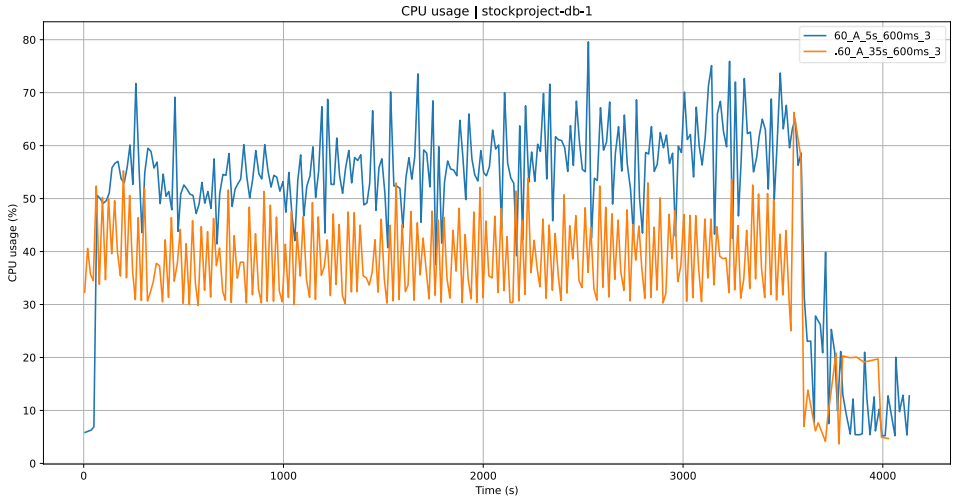
Na rys. 4.5 zaprezentowano zmiany w czasie odpowiedzi API dla dwóch różnych ustawień interwału pomiędzy transakcjami, wynoszących odpowiednio pięć oraz 35 sekund. Z analizy wykresu wynika, że w przypadku krótszego interwału czas odpowiedzi API był znacząco wyższy w porównaniu do ustawienia dłuższego interwału. Taki wzrost może być wynikiem większego obciążenia systemu, które pojawia się przy częstszych transakcjach. Na Rys. 4.6 przedstawiono zużycie procesora bazy danych giełdy dla testu z czasem transakcji 5s oraz 35s. To jedyny kontener, który wykazał różnice w wykorzystaniu przez procesor. Wyraźnie widać, że przy krótkim czasie transakcji baza danych jest obciążona o około 10% więcej przez cały czas trwania testu. Zachowanie dla innych czasów transakcji było bardzo podobne do tego z testu 11. Warto zauważyć, w testach 10 i 12 obserwowano analogiczne zachowanie jak dla testu 11.



Rys. 4.4 Porównanie średnich czasów transakcji – [test 9, test 10, test 11, test 12]

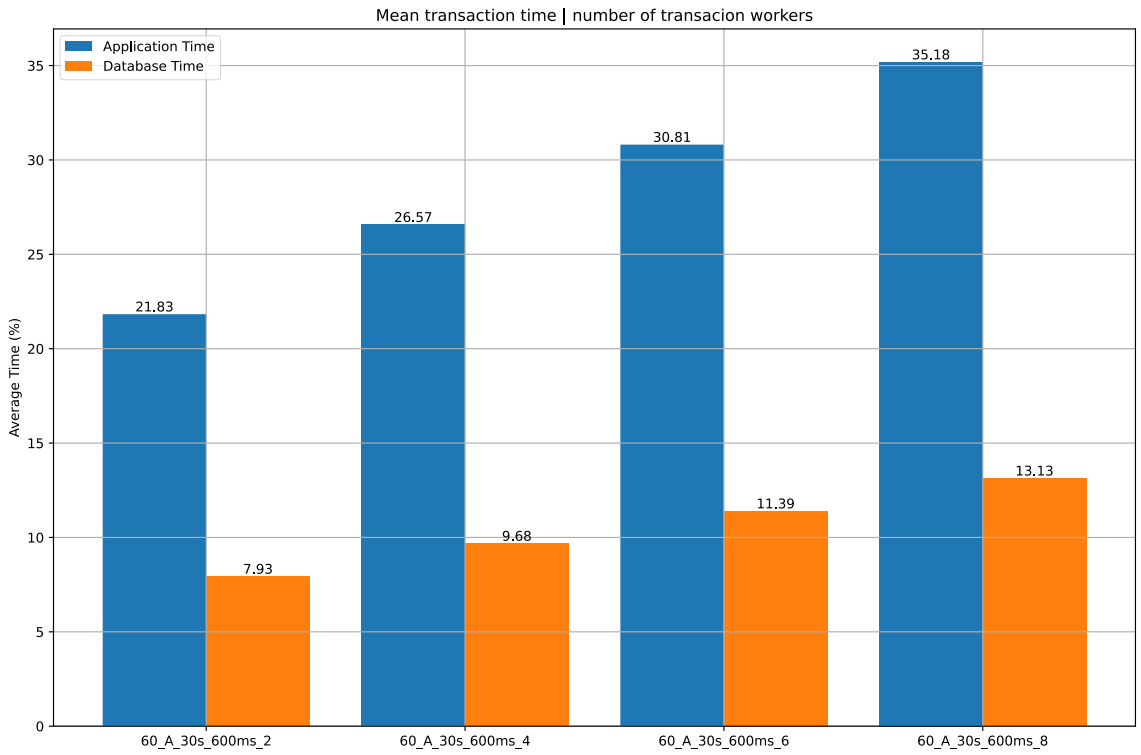


Rys. 4.5 Zestawienie czasu API – [test 9,test 11]

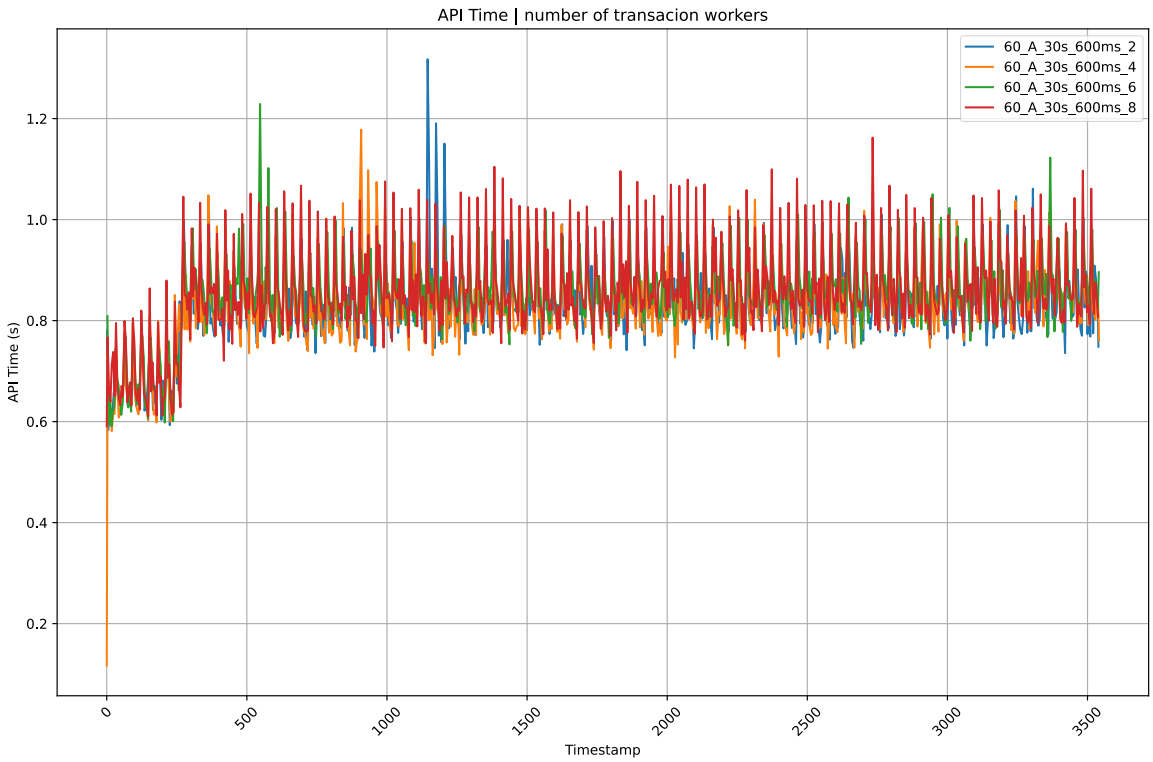


Rys. 4.6 Zużycie procesora bazy danych giełdy – [test 9, test 11]

Na Rys. 4.7 i Rys. 4.8 widać, że zwiększanie liczby kontenerów do przeprowadzania transakcji wpływa negatywnie zarówno na czas API jak i czas realizacji transakcji. W transakcjach wyraźnie rośnie czas aplikacji oraz czas bazy danych. Po pobieżnej analizie wykresu z czasami API można mylnie dojść do wniosku, że wykresy są w zasadzie takie same. Jednak różni je jeden bardzo ważny element. Widoczne okresowe skoki podczas przeprowadzania transakcji w większości przypadków są bardziej gwałtowne, gdy liczba kontenerów realizujących transakcje jest większa.



Rys. 4.7 Porównanie średnich czasów transakcji – [test 13, test 14, test 15, test 16]



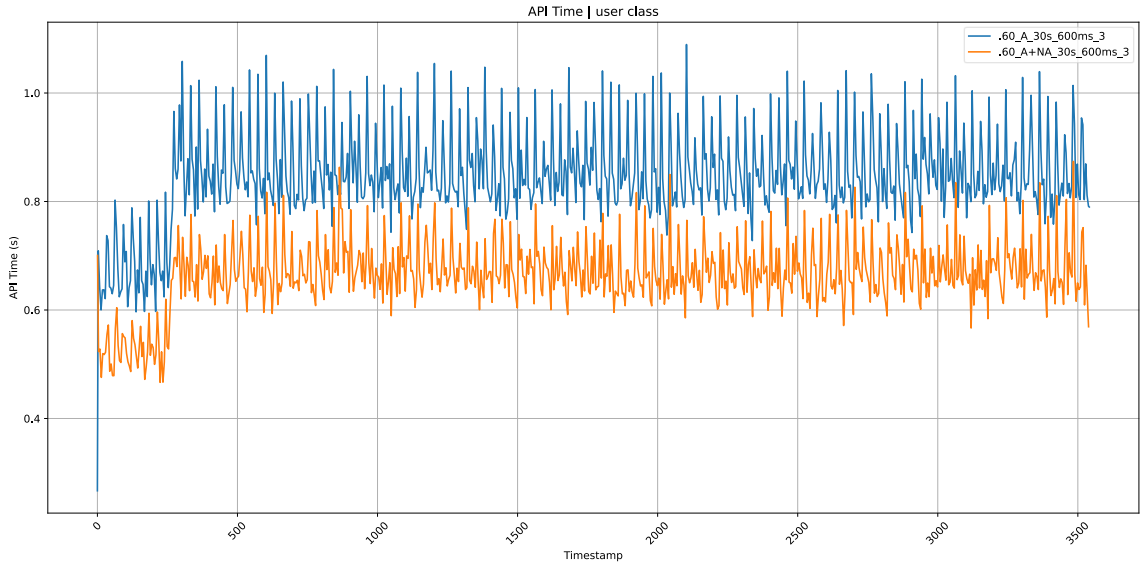
Rys. 4.8 Zestawienie czasu API – [test 13, test 14, test 15, test 16]

W Ostatniej analizowanej grupie testów zmienianym parametrem są użyte klasy użytkowników. Test 19 w którym brało udział 40 graczy aktywnych i 20 przeglądających

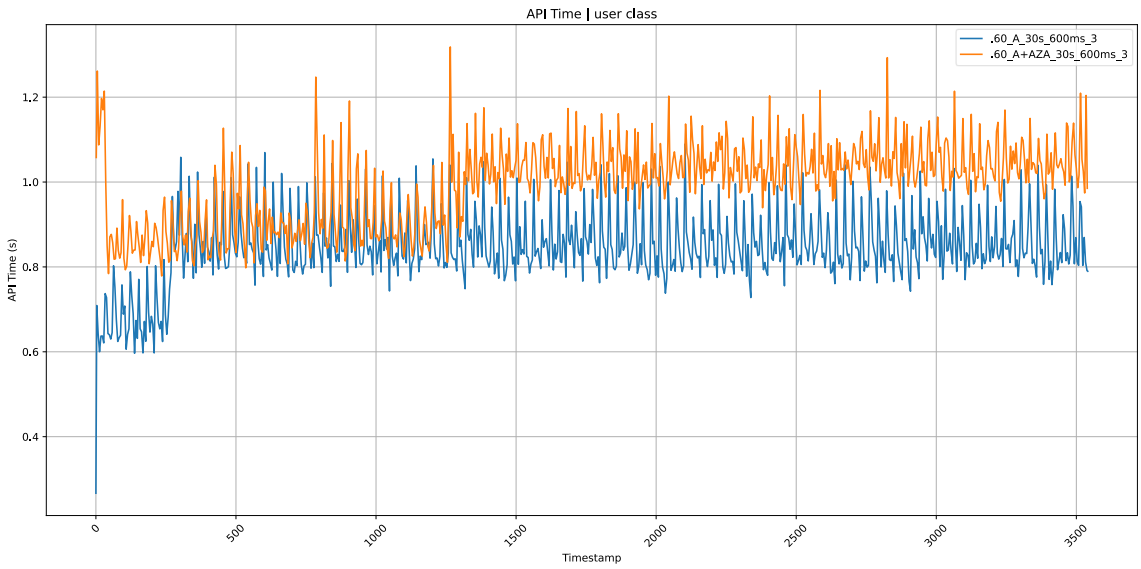
miał mniejszy czas API od testu 17, gdzie wszyscy użytkownicy byli aktywni. Różnica przez cały czas trwania testu utrzymywała się na podobnym poziomu i wynosiła około 0.15s. Opisaną zależność zaprezentowano na Rys. 4.9.

Na Rys. 4.10 przedstawiono wyniki czasu API dla testu 17 i testu 20. Jak widać gracze z analizą trendów cen akcji przez znaczną część testu osiągali dłuższe czasu odpowiedzi. Od około 300 do około 1200 sekundy test składający się z samych aktywnych graczy, dawał zbliżone wyniki do testu 20. Nie udało się ustalić przyczyny takiego zachowania. Na Rys. 4.11 przedstawiającym zużycie procesora aplikacji giełdy można zauważyć korelację: dla testu 20 gdy czas API miał wartość od 0.8s do 1s poza małymi skokami to wykorzystanie procesora było na wyższym poziomie, a gdy czas API był powyżej jednej sekundy to zasoby procesora były zużywane w mniejszym stopniu. W pozostałych testach zachowanie, w którym mniejszy czas API oznacza wyższe zużycie procesora, a wyższy czas API mniejsze, jest również zauważalne. Wykorzystanie zasobów procesora w pozostałych kontenerach było w testach od 17 do 20 takie same.

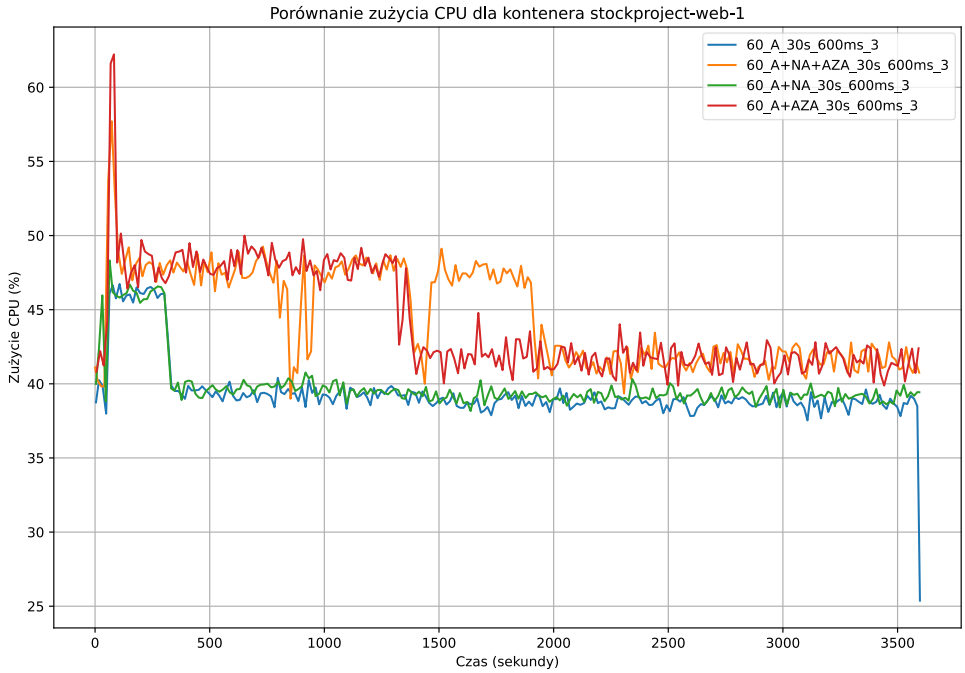
Z analizy Rys. 4.12 można wywnioskować, że realizacja transakcji w teście 19 była prawie 2 razy krótsza od tej z testu 17. Wynika to z tego, że 33% graczy nie miało żadnego wpływu na transakcje. Po porównaniu czasu transakcji w teście 19 i 20 można zauważyć, że użytkownik z analizą trendu cenowego w niewielkim stopniu zwiększa czas transakcji. 30 użytkowników z analizą dało większy czas o około dwie sekundy od 10 aktywnych użytkowników. Oznacza to, że przy losowym zachowaniu aktywnych użytkowników występowanie trendów akcji firm jest rzadkie.



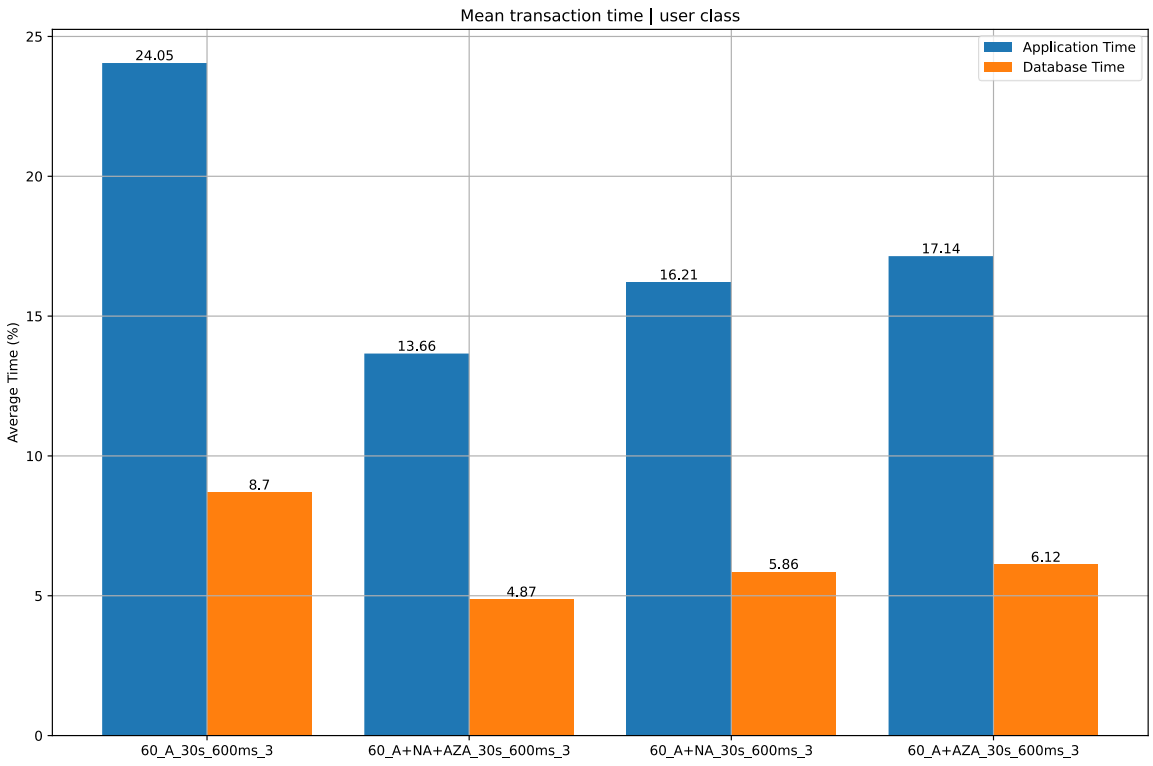
Rys. 4.9 Zestawienie czasu API – [test 17, test 19]



Rys. 4.10 Zestawienie czasu API – [test 17, test 20]



Rys. 4.11 Zużycie procesora aplikacji giełdy – [test 17, test 18, test 19, test 20]



Rys. 4.12 Porównanie średnich czasów transakcji – [test 17, test 18, test 19, test 20]

4.4 Analiza z użyciem reguł asocjacyjnych

Przeprowadzono analizę reguł asocjacyjnych dla wszystkich testów. Uznano, że reguły są istotne gdy, ich wsparcie jest większe niż 0.3, ufność jest większa niż 0.8 oraz podniesienie jest większe od 1.2. Ustawienie mniejszego minimalnego wsparcia daje więcej reguł asocjacyjnych, ale postanowiono że nie będą brane pod uwagę.

W testach 1, 3, 4, 8, 9, 10 znaleziono dwie reguły, które przedstawia Tabela 4.3. Poszczególne wartości wsparcia, ufności i podniesienia różnią się nieznacznie pomiędzy testami. Jak widać wysoki czas aplikacji powoduje wysoki czas bazy danych. Odwrotna zależność również jest prawdziwa. Wyższa ufność zachodzi, gdy przesłanką jest wysoki czas aplikacji, a konsekwencją jest wysoki czas bazy danych.

Tabela 4.3 Reguły asocjacyjne dla testu nr 1.

antecedents	consequents	support	confidence	lift
high_application_time	high_database_time	0.47	0.93	1.68
high_database_time	high_application_time	0.47	0.84	1.68

W pozostałych testach jedynie test 18, zawierający trzy różne klasy użytkownika, dał więcej reguł asocjacyjnych spełniających wyżej wymienione kryteria. Tabela 4.4 pokazuje uzyskane reguły. Wszystkie reguły zawierają wysokie zużycia procesora danego kontenera

jako przesłankę oraz konsekwencję. W niektórych regułach pojawia się wysoki czas bazy danych. Pierwsze sześć reguł zawiera w różnych konfiguracjach wysokie zużycie procesora bazy danych testu, wysokie zużycie procesora giełdy oraz wysoki czas bazy danych. Wszystkie te reguły mają takie samo wsparcie, ufność i podniesienie.

Z kolei w regułach od siódmej do dwunastej oprócz wyżej wspomnianych właściwości jest też wysokie zużycie procesora bazy danych giełdy. W tym zestawie reguł wsparcie jest takie samo i wynosi 0.34. Wartości ufności i podniesienia są podobne, gdy ufność była większa to podniesienie również. Wśród znalezionych reguł asocjacyjnych dane właściwości powtarzają się jako przesłanka, a następnie konsekwencja tworząc wzajemne zależności i z uwagi na to trudno jest wyciągnąć wnioski.

Tabela 4.4 Reguły asocjacyjne dla testu nr 18.

antecedents	consequents	support	confidence	lift
high_cpu_usage_db_test	high_cpu_usage_web	0.41	0.82	1.64
high_cpu_usage_web	high_cpu_usage_db_test	0.41	0.82	1.64
high_cpu_usage_db_test, high_database_time	high_cpu_usage_web	0.41	0.82	1.64
high_cpu_usage_web, high_database_time	high_cpu_usage_db_test	0.41	0.82	1.64
high_cpu_usage_db_test	high_cpu_usage_web, high_database_time	0.41	0.82	1.64
high_cpu_usage_web	high_cpu_usage_db_test, high_database_time	0.41	0.82	1.64
high_cpu_usage_db_test, high_cpu_usage_db	high_cpu_usage_web	0.34	0.89	1.79
high_cpu_usage_db, high_cpu_usage_web	high_cpu_usage_db_test	0.34	0.89	1.78
high_cpu_usage_db_test, high_cpu_usage_web	high_cpu_usage_db	0.34	0.82	1.63
high_cpu_usage_db_test, high_cpu_usage_db	high_cpu_usage_web, high_database_time	0.34	0.89	1.79
high_cpu_usage_db, high_cpu_usage_web	high_cpu_usage_db_test, high_database_time	0.34	0.89	1.78
high_cpu_usage_db_test, high_cpu_usage_web	high_cpu_usage_db, high_database_time	0.34	0.82	1.63

4.5 Analiza z użyciem klasyfikatorów

Uznano, że testy zawierające różne klasy użytkownika dały wyniki, które warto poddać dalszej analizie. W przeprowadzonych testach tylko jeden miał trzy klasy użytkowników. Zdecydowano, że do badań konieczna jest większa liczba testów, których są wszystkie klasy użytkowników. Tabela 4.5 przedstawia dodatkowe testy. Test 27 ma parametry, w których system zachowywał się najbardziej stabilnie, czyli czas pomiędzy transakcjami, czas pomiędzy kolejnymi zapytaniami oraz liczba kontenerów realizująca transakcje w mniejszym stopniu niż reszta testów obciążała system. W pozostałych testach

wybrano taki zestaw parametrów, który wyróżniał się w analizie podstawowej. Parametry w teście 21 są na podstawie testu 1, dla testu 22 na podstawie testu 4, dla testu 23 na podstawie testu 5, dla testu 24 na podstawie testu 8, dla testu 25 na podstawie testu 9, dla testu 26 na podstawie testu 12, dla testu 27na podstawie testu 13, dla testu 28 na podstawie testu 16.

Tabela 4.5 Opis dodatkowych wykonanych testów

Nr testu	A	NA	AZA	Ttr[s]	Trq[ms]	Ktr
Test 21	4	2	4	30	600	3
Test 22	52	26	52	30	600	3
Test 23	24	12	24	30	200	3
Test 24	24	12	24	30	1100	3
Test 25	24	12	24	5	600	3
Test 26	24	12	24	50	600	3
Test 27	24	12	24	30	600	2
Test 28	24	12	24	30	600	8

W klasyfikacji wypróbowano następujące modele:

1) Xgb.
2) ExtraTreesClassifier.
3) DecisionTreeClassifier.
4) KNeighborsClassifier.
5) RandomForestClassifier.
6) LinearSVC
7) MLPClassifier.
8) ExtraTreeClassifier.
9) BernoulliNB.
10) GaussianNB.
11) NearestCentroid.
12) RadiusNeighborsClassifier.
13) LinearDiscriminantAnalysis.
14) QuadraticDiscriminantAnalysis.

Najlepsze wyniki klasyfikacji uzyskano dla testu 22. Spośród wyżej wymienionych metod pierwsze siedem bez dostosowania hiper parametrów [20] osiągnęło dokładność powyżej 65%. Pozostałe radziły sobie dużo gorzej. Z tego powodu tylko te algorytmy zostały wzięte pod uwagę do dalszej klasyfikacji. Tabela 4.6 zwiera osiągnięte dokładności dla poszczególnych testów. Z uwagi na długi czas potrzebny do znalezienia najlepszych hiper

parametrów oraz niewielką poprawę wyników, do klasyfikacji zastosowano modele z parametrami, które osiągnęły największą dokładność dla testu 22. Jak widać klasyfikatory z rodziny drzew decyzji miały dla każdego testu zbliżone do siebie wyniki i osiągały zadowalające wyniki w porównaniu do innych. Na przestrzeni wszystkich testów najwyższą dokładność dał model Multi-layer Perceptron, choć różnice między nim, a algorytmami drzew decyzji był w granicy jednego procenta, a czas trenowania MLP był o wiele dłuższy.

Tabela 4.6 Dokładność klasyfikacji klas użytkownika przez wybrane algorytmy.

Nr testu / Algorytm	Test 21	Test 22	Test 23	Test 24	Test 25	Test 26	Test 27	Test 28
Xgb	74%	79%	72%	73%	74%	72%	72%	73%
ExtraTreesClassifier	73%	79%	72%	74%	74%	72%	73%	73%
DecisionTreeClassifier	74%	79%	72%	73%	74%	72%	72%	73%
KNeighborsClassifier	72%	76%	68%	71%	71%	69%	69%	67%
RandomForestClassifier	73%	79%	72%	74%	74%	72%	73%	73%
LinearSVC	62%	73%	63%	68%	68%	65%	66%	67%
MLPClassifier	74%	79%	72%	74%	74%	72%	73%	73%

Na Rys. 4.13 zaprezentowano macierz pomyłek dla testu nr 22, w którym zastosowano klasyfikator MLPClassifier. Wyniki przedstawione w tej macierzy dostarczają wglądu w szczegółową charakterystykę klasyfikacji dla trzech klas użytkowników: WebsiteActiveUser, WebsiteActiveUserWithMarketAnalyze oraz WebsiteReadOnlyUser.

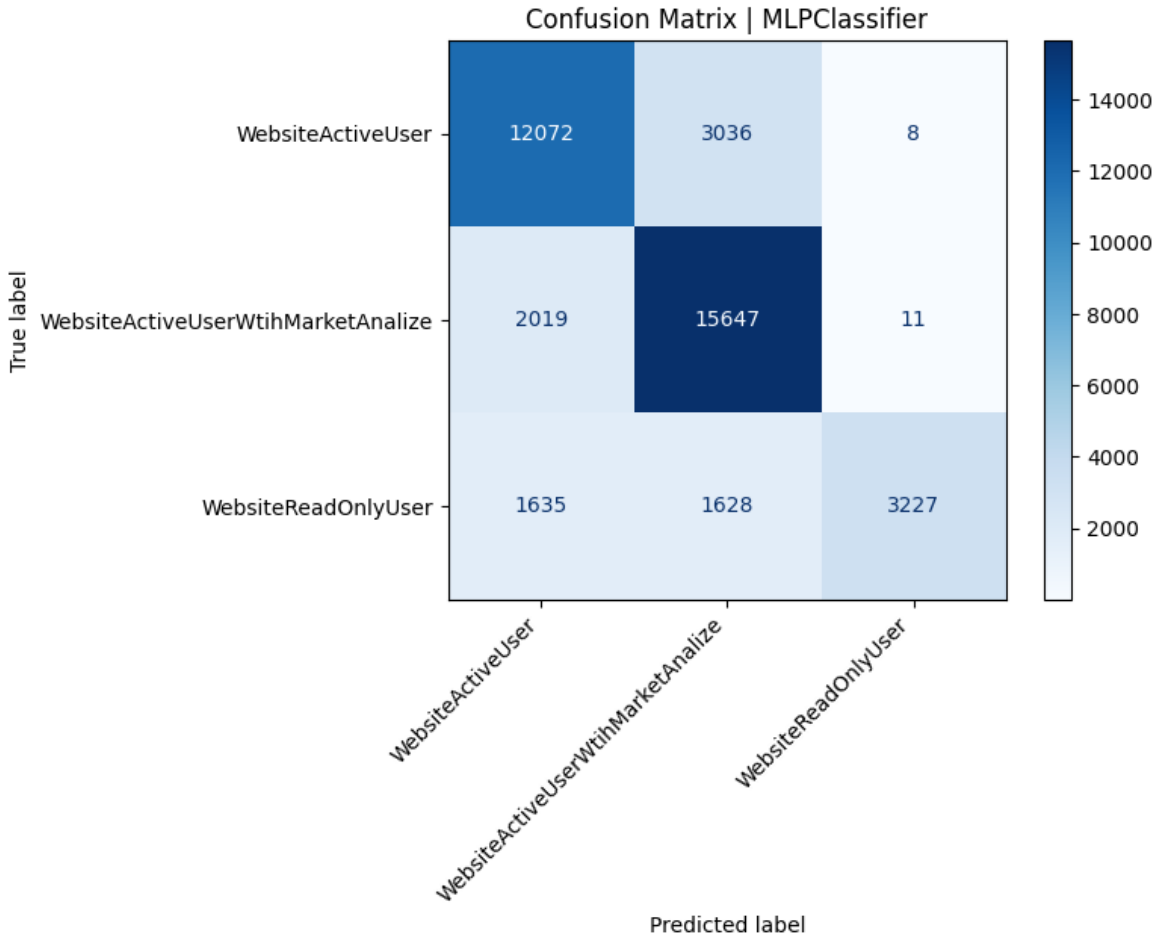
Dla klasy WebsiteReadOnlyUser, choć model osiąga precyzję na poziomie 99%, zauważalny jest problem z czułością. Znacząca część obserwacji, które faktycznie należą do tej klasy, jest błędnie przypisywana do innych kategorii (WebsiteActiveUser oraz WebsiteActiveUserWithMarketAnalyze). W praktyce oznacza to, że model często klasyfikuje użytkowników pasywnych, ograniczających się do samego odczytu strony, jako użytkowników aktywnych lub takich, którzy dodatkowo dokonują analiz rynkowych. Tego rodzaju pomyłki skutkują niską czułością dla klasy WebsiteReadOnlyUser, która wynosi 50%.

Dalsza analiza wyników w macierzy pomyłek pokazuje także trudności klasyfikatora z rozróżnieniem klasy WebsiteActiveUser oraz WebsiteActiveUserWithMarketAnalyze. Warto zauważyć, że istotna liczba obserwacji z klasy WebsiteActiveUser jest klasyfikowana jako WebsiteActiveUserWithMarketAnalyze. Może to sugerować, że model nie jest w stanie w pełni uchwycić subtelnych różnic między użytkownikami aktywnie korzystającymi

z witryny a tymi, którzy dodatkowo analizują trendy rynkowe. Tego typu błędy mogą wynikać z podobnych wzorców interakcji użytkowników w tych dwóch klasach.

Kolejnym istotnym czynnikiem wpływającym na wyniki klasyfikacji jest niezbilansowanie klas w zbiorze danych. Z analizy macierzy pomyłek wynika, że liczebność poszczególnych klas nie jest jednakowa. O ile klasa WebsiteActiveUserWithMarketAnalyze jest podobnie liczna jak WebsiteActiveUser to WebsiteReadOnlyUser jest dużo rzadsza. W klasyfikacji niezbilansowanej model często faworyzuje większą klasę, co prowadzi do sytuacji, w której obserwacje z mniejszych klas są błędnie przypisywane do większej klasy dominującej.

Ogólnie rzecz biorąc, z przetestowanych metod MLPClassifier charakteryzuje się najlepszą dokładnością klasyfikacji. Jednakże, występowanie licznych błędów przy klasyfikacji klasy WebsiteReadOnlyUser wskazuje na potrzebę dalszej analizy i ewentualnej poprawy parametrów modelu. Macierz pomyłek dla pozostałych metod wygląda bardzo podobnie.



Rys. 4.13 Macierz pomyłek dla testu nr 22 i modelu MLPClassifier

Bazując na wynikach uzyskanych w innych pracach, które również wykorzystywały klasyfikację, spodziewano się, że uzyskane dokładności będą wyższe. Podjęto się klasyfikacji klas użytkownika, a w przytoczonych pracach skupino się na klasyfikacji endpointów oraz przewidywanie zużycia zasobów. Okazało się, że wykrywanie klas użytkownika jest trudniejszym zadaniem [10,21].

5 Podsumowanie

Niniejsza praca magisterska przedstawia kompleksowe podejście do projektowania, implementacji i testowania aplikacji giełdowej oraz narzędzia obciążającego. Celem pracy było zbadanie wydajności systemu giełdowego w warunkach intensywnego obciążenia oraz ocena jego niezawodności i skalowalności.

W ramach pracy zrealizowano następujące zadania:

1) Projekt aplikacji giełdowej – opracowano aplikację z funkcjonalnościami zarządzania ofertami kupna i sprzedaży, przeprowadzania transakcji oraz cyklicznej aktualizacji danych. System został zaprojektowany z uwzględnieniem nowoczesnych technologii, takich jak Django, PostgreSQL, Celery, Redis, Locust oraz Docker.
2) Tworzenie narzędzia obciążającego – opracowano aplikację symulującą zachowanie różnych typów użytkowników giełdy, co pozwoliło na dokładne testy wydajnościowe.
3) Eksperymenty i analiza wyników – przeprowadzono liczne testy obciążeniowe, zróżnicowane pod względem liczby użytkowników, częstotliwości operacji, dostępnych zasobów oraz klas użytkownikow. Zastosowano zaawansowane metody analizy, takie jak reguły asocjacyjne i klasyfikatory.

Z przeprowadzonych badań wynika, że:

1) Nie zauważono wpływu zużycia pamięci RAM na wydajność systemu.
2) Różnica czasów API dla poszczególnych endpointów jest niewielka.
3) Większa liczba użytkowników skutkowała wyższym czasem API.
4) Test gdzie liczba użytkownika to 10 miał niższe zużycie procesora.
5) Im większy czas pomiędzy kolejnymi zapytaniami, tym czas oczekiwania na odpowiedź jest mniejszy.
6) Bardzo częste transakcje skutkują zwiększonym zużyciem procesora bazy danych giełdy, co zwiększa czas API.
7) Dodanie 2 kontenera do realizacji transakcji zwiększyło efektywność przeprowadzania transakcji. Natomiast dodawanie kolejnych kontenerów do realizacji transakcji nie dawało korzyści, a nawet skutkowało małym obniżeniem wydajności.
8) Gdy część graczy tylko przeglądała, to czas API zmniejszył się.

9) Wprowadzenie użytkownika z analizą trendów zwiększyła obciążenie procesora aplikacji giełdy, a przez to zwiększył się czas API.
10) Aktywni użytkownicy dodający losowo oferty dość rzadko generowali trendy w kursach akcji firm. Co skutkowało tym, że użytkownicy z analizą rynku generowali niewiele ofert kupna i sprzedaży.
11) Z pomocą reguł asocjacyjnych nie udało się znaleźć istotnych powiązań.
12) Klasyfikatory z rodziny drzew decyzji miały najlepsze wyniki biorąc pod uwagę dokładność oraz czas potrzebny na wytrenowanie modelu.
13) Klasyfikator MLP był najlepszy biorąc pod uwagę samą dokładność. Jednak różnica to jedynie jeden procent.
14) Najlepsza uzyskana dokładność dla rozpoznania klasy użytkownika to 79%. Udało się ją uzyskać tylko dla jednego testu.
15) Macierz pomyłek pokazuje, że klasyfikator dość często myli użytkowników aktywnych z użytkownikami z analizą trendów. Czyli dla klasyfikatora te klasy są do siebie podobne.
16) Precyzja dla klasy użytkownika przeglądającego jest wręcz idealna. Natomiast dokładność jest bardzo niska, przez to, że jest duży problem z czułością.

Podczas realizacji pracy zidentyfikowano obszary, które można by było lepiej rozwiązać:

1) Wprowadzenie bardziej zaawansowanych mechanizmów synchronizacji w celu uniknięcia konfliktów podczas jednoczesnych aktualizacji sald użytkowników.
2) Rozbudowanie modułu analitycznego o zaawansowane metody predykcyjne, które mogłyby wspomagać decyzje użytkowników giełdy.
3) Lepsze dostosowanie algorytmów podejmowania decyzji przez użytkowników z analizą rynku, aby bardziej odróżniali się od aktywnych użytkowników.

Tematyka pracy otwiera możliwości dalszych badań i rozwoju, takich jak:

1) Implementacja algorytmów sztucznej inteligencji do przewidywania trendów rynkowych.
2) Analiza skalowalności systemu przy znacznie większej liczbie użytkowników i operacji.
3) Badanie wpływu różnych modeli bazy danych na wydajność aplikacji giełdowej.

Autor za własny wkład pracy uważa:

1) Przegląd aktualnego stanu wiedzy w temacie wydajności podobnych systemów giełd.
2) Opracowanie pełnej architektury systemu giełdowego oraz aplikacji obciążającej.
3) Przeprowadzenie kompleksowych testów wydajnościowych.
4) Przygotowanie skryptów przygotowujących dane do analizy oraz realizujące analizę danych.
5) Interpretację wyników na podstawie zaawansowanych metod analitycznych.
6) Sformułowanie wniosków i rekomendacji dotyczących projektowania wydajnych systemów transakcyjnych

Literatura

[1] Gomber P., Koch J.-A., Siering M.: Digital Finance and FinTech: current research and future research directions. Journal of Business Economics, Springer, vol. 87(5), pages 537-580, July 2017.

[2] Shklar L., Rosen R.: Web Application Architecture: Principles, Protocols and Practices. Wiley, 2009.

[3] Dokumentacja Django, https://docs.djangoproject.com/en/5.1/ dostęp 27.11.2024.

[4] Dokumentacja PostgreSQL, https://www.postgresql.org/docs/ dostęp 27.11.2024.

[5] Dokumentacja Celery, https://docs.celeryq.dev/en/stable/index.html dostęp 27.11.2024.

[6] Dokumentacja Locust, https://docs.locust.io/en/stable/ dostęp 27.11.2024.

[7] Dokumentacja Dockera, https://docs.docker.com/manuals/ dostęp 27.11.2024.

[8] Drabek J. Kod źródłowy aplikacji giełdy, https://github.com/jasiekdrabek/stockProject dostęp 21.11.2024.

[9] Piszko R.: Badanie wydajności środowiske-commerce w postaci benchmarku, Praca magisterska, Politechnika Rzeszowska, 2022.

[10] Pawłoś Ł.: Badanie logów webowego środowiska produkcyjnego z wykorzystaniem technik uczenia maszynowego, Praca magisterska, Politechnika Rzeszowska, 2024.

[11] Kantardzic, M.: Data Mining (Concepts, Models, Methods, and Algorithms), pages 335-356, Wiley,2019.

[12] Klasyfikacja, https://www.geeksforgeeks.org/getting-started-with-classification/ dostęp 27.11.2024

[13] Metryki w klasyfikacji, https://www.geeksforgeeks.org/evaluation-metrics-for-classification-model-in-python/ dostęp 27.11.2024.

[14] Dokumentacja Pandas, https://pandas.pydata.org/docs/ dostęp 27.11.2024.

[15] Dokumentacja Matplotlib, https://matplotlib.org/stable/index.html dostęp 27.11.2024.

[16] Dokumentacja Scikit-learn, https://scikit-learn.org/stable/supervised_learning.html dostęp 27.11.2024.

[17] Mlxtend, https://rasbt.github.io/mlxtend/USER_GUIDE_INDEX/#frequent_patterns dostęp 27.11.2024.

[18] Xgboost, https://xgboost.readthedocs.io/en/latest/python/python_intro.html dostęp 27.11.2024.

[19] Drabek J. Skrypty do analizy danych, https://github.com/jasiekdrabek/testAnalysis dostęp 21.11.2024.

[20] Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A.-L., Deng, D., & Lindauer, M. (2023). Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. WIREs Data Mining and Knowledge Discovery, 13(2), e1484. https://doi.org/10.1002/widm.1484

[21] Borowiec M.: Badanie logów z wykorzystaniem analizy danych i uczenia maszynowego, Praca magisterska, Politechnika Rzeszowska, 2023.

Sygnatura:

POLITECHNIKA RZESZOWSKA im. I. Łukasiewicza Rzeszów, 2024
Wydział Elektrotechniki i Informatyki

STRESZCZENIE PRACY DYPLOMOWEJ MAGISTERSKIEJ

PROJEKT APLIKACJI GIEŁDY INTERNETOWEJ ORAZ APLIKACJI OBCIĄŻAJĄCEJ

Autor: Jan Drabek, nr albumu: EF/I-DU-175304
Opiekun: dr inż. Tomasz Rak
Słowa kluczowe: giełda internetowa, testy wydajnościowe, Django, Docker, klasyfikacja danych.

Praca magisterska dotyczy zaprojektowania, implementacji oraz analizy wydajności systemu giełdowego w warunkach wysokiego obciążenia. W ramach pracy opracowano aplikację giełdową opartą na Django, wykorzystującą Celery i Redis do przetwarzania zadań cyklicznych oraz framework Locust do symulacji aktywności użytkowników. System został przetestowany w środowisku kontenerowym z użyciem Docker Compose. Przeprowadzono liczne eksperymenty, które pozwoliły na ocenę wpływu parametrów, takich jak liczba użytkowników oraz czasy między transakcjami na czasy odpowiedzi API i zużycie zasobów. Do analizy wykorzystano reguły asocjacyjne oraz klasyfikację.

RZESZOW UNIVERSITY OF TECHNOLOGY Rzeszow, 2024
Faculty of Electrical and Computer Engineering

DIPLOMA THESIS (MS) ABSTRACT

DESIGN OF AN ONLINE STOCK EXCHANGE APPLICATION AND A LOAD TESTING TOOL

Author: Jan Drabek, code: EF/I-DU-175304
Supervisor: dr inż. Tomasz Rak
Key words: online stock exchange, performance testing, Django, Docker, data classification.

This master's thesis focuses on the design, implementation, and performance analysis of a stock exchange system under high load conditions. The work involved the development of a stock exchange application based on Django, utilizing Celery and Redis for processing periodic tasks, and the Locust framework for simulating user activity. The system was tested in a containerized environment using Docker Compose.
Numerous experiments were conducted to measure the impact of parameters such as the number of users and transaction times on API response times and resource consumption. The analysis utilized association rules and data classification.